

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Вятский государственный университет»

Колледж ВятГУ

ДОПУСКАЮ К ЗАЩИТЕ:
Заместитель директора колледжа
по учебной работе

(подпись) Жвакина С.Г.
(Ф.И.О.)

«19» _____ июня _____ 2023 г.

ДИПЛОМНЫЙ ПРОЕКТ

На тему:

Разработка мобильного приложения для развития концентрации внимания

Студент

Владимиров Кирилл Сергеевич

(Фамилия, Имя, Отчество)

(подпись)

(дата)

Программа подготовки специалистов среднего звена по специальности

09.02.07 Информационные системы и программирование

(шифр и наименование специальности)

Форма обучения

очная

Руководитель Самоделкин Павел

Андреевич

(Ф.И.О.)

(подпись)

Киров, 2023

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Вятский государственный университет»
Колледж ВятГУ

УТВЕРЖДАЮ

_____ Г.А. Чистяков
«10» _____ апреля 2023 г.

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ
для выполнения дипломного проекта

Специальность 09.02.07 Информационные системы и программирование

Группа ИСПк-403-52-00 Ф.И.О. студента Владимиров Кирилл Сергеевич

Тема дипломного проекта:

Разработка мобильного приложения для развития концентрации внимания

Дата выдачи задания: «10» _____ апреля 2023 г.

Работа должна быть сдана не позднее «19» _____ июня 2023 г.

Исходные данные для выполнения дипломного проекта:

1. Документация по языкам программирования Python, JavaScript, React
2. Исследования нейробиологии внимания, когнитивных стратегий улучшения концентрации и влияние отвлекающих факторов на эффективность внимания
3. Методические указания по улучшению концентрации внимания и ее эффективного использования

Перечень подлежащих разработке вопросов:

1. Анализ предметной области мобильных приложений по развитию концентрации внимания, обзор, существующий аналогов
2. Проектирование архитектуры системы; проектирование структуры приложения, сервера и интерфейса; разработка алгоритмов функционирования приложения
3. Реализация мобильного приложения

Перечень графических материалов:

Иллюстративный материал в форме презентации.

Руководитель дипломного проекта: _____ / _____ /
(подпись) (Ф.И.О. руководителя)

Задание принял к исполнению _____ / «10» апреля 2023 г.
(подпись) (Ф.И.О. студента)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Вятский государственный университет»

Колледж ВятГУ

УТВЕРЖДАЮ

_____ Г.А. Чистяков
«31» _____ марта 2023 г.

КАЛЕНДАРНЫЙ ГРАФИК ВЫПОЛНЕНИЯ
ДИПЛОМНОГО ПРОЕКТА

Специальность Информационные системы и программирование
Группа ИСПк-403-52-00 Ф.И.О. Владимиров Кирилл Сергеевич
_____ студента _____
Тема дипломного проекта:
Разработка мобильного приложения для развития концентрации внимания

Подготовка дипломного проекта	Сроки
Утверждение задания на дипломный проект	10.04.2023
Представление программного продукта руководителю	29.05.2023
Представление пояснительной записки руководителю	12.06.2023
Устранение замечаний, оформление и представление руководителю полного текста работы	13.06.2023 – 14.06.2023
Получение отзыва руководителя дипломного проекта	15.06.2023 – 17.06.2023
Подготовка тезисов выступления и презентации	15.06.2023 – 17.06.2023
Представление дипломного проекта нормоконтролеру	15.06.2023 – 17.06.2023
Срок сдачи студентом законченного дипломного проекта заместителю директора (по учебной работе)	19.06.2023

Руководитель дипломного проекта _____ / _____ «10» апреля 2023
г.
подпись ФИО

Задание принял к исполнению _____ / _____ «10» апреля 2023 г.
подпись ФИО

РЕФЕРАТ

ВЛАДИМИРОВ К.С., РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ
ДЛЯ РАЗВИТИЯ КОНЦЕНТРАЦИИ ВНИМАНИЯ. ТПЖА 09.02.07.695577
ПЗ: Дипломный проект / Колледж ВятГУ, Самоделкин П.А. – Киров, 2023. –
ПЗ 61 с., 11 источников.

МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, ИГРЫ С КОНЦЕНТРАЦИЕЙ
ВНИМАНИЯ, АВТОМАТИЗАЦИЯ, КЛИЕНТ-СЕРВЕР.

Объект исследования – мобильное приложение.

Предмет исследования – информационная система.

Цель дипломного проекта – разработать приложение для развития и
получения навыков концентрации внимания.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБЗОР АНАЛОГОВ	5
1.1 Анализ предметной области	5
1.2 Обзор существующих аналогов.....	7
1.3 Анализ проблемной области.....	14
2 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ	17
2.1 Архитектура системы.....	17
2.2 Проектирование структуры ПО.....	19
2.3 Проектирование структуры сервера.....	22
2.4 Проектирование интерфейса	25
2.5 Разработка алгоритмов функционирования ПО	28
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ.....	32
3.1 Выбор и основания средств разработки.....	32
3.2 Разработка клиентской части.....	41
3.3 Разработка серверной части.....	49
3.4 Тестирование программного продукта	54
ЗАКЛЮЧЕНИЕ	59
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	60
ПРИЛОЖЕНИЕ А	62

					<i>ТПЖА.09.02.07.695577 ПЗ</i>		
Изм.	Лист	Фамилия	Подпись	Дата			
Разраб.		Владимиров К.С.			Разработка мобильного приложения для развития концентрации внимания	Лит.	Лист
Провер.		Самоделкин П.А.					2
Реценз							61
Н. Контр.		Лазарева О.В.				Колледж ВятГУ	
Утверд.							

ВВЕДЕНИЕ

В последнее время наблюдается впечатляющий рост технологий и информационных систем, которые стали неотъемлемой частью нашей жизни. Это не только упрощает нашу повседневную деятельность, но также создает новые возможности для бизнеса, науки и других областей человеческой жизни. Однако, в то же время, мы обнаруживаем, что все большее количество людей испытывает проблему концентрации внимания. Рассеянность и отсутствие фокуса стали серьезными проблемами, которые в особенности изучаются в психологии и нейронауке.

Создание мобильных приложений, способных помочь пользователям улучшить концентрацию внимания, является важной и актуальной темой в настоящее время. Благодаря использованию новейших технологий, разработчики могут создавать мобильные приложения, которые позволяют пользователю стимулировать свой мозг и повышать когнитивные способности.

Одним из основных элементов, которые помогают повысить концентрацию внимания, является использование техники свободного дыхания, а также избегание сильных перепадов настроения и стрессовых ситуаций. Эти элементы могут быть включены в различные мобильные приложения, которые помогают управлять концентрацией внимания, улучшают здоровье и повышают работоспособность.

Предмет исследования – информационная система.

Объект исследования – мобильное приложение

Цель квалификационной работы: разработать приложение для развития и получения навыков концентрации внимания.

					<i>ТПЖА.09.02.07.695577 ПЗ</i>	Лист
						3
Изм.	Лист	№ докум.	Подпись	Дата		

Задачи квалификационной работы:

- 1) Изучить предметную область;
- 2) Выбрать среду разработки приложения;
- 3) Изучить структуру строения информационной системы;
- 4) Проанализировать существующие информационные системы на рынке;
- 5) Спроектировать информационную систему;
- 6) Разработать информационную систему.

Методологической базой исследования является:

- 1) учебные ресурсы;
- 2) статьи в сети Internet;
- 3) книги по программированию.

Практическая значимость – разработка мобильного приложения, которое обеспечит возможность развития навыков концентрации внимания.

					<i>ТПЖА.09.02.07.695577 ПЗ</i>	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБЗОР АНАЛОГОВ

1.1 Анализ предметной области

В настоящее время существует множество мобильных приложений, которые предлагают пользователю методы и техники для улучшения концентрации внимания. Например, приложения, с помощью которых можно отслеживать время, проведенное в социальных сетях или других отвлекающих приложениях, а также приложения с оповещениями, которые уведомляют пользователя о необходимости вернуться к работе после предыдущего перерыва.

Некоторые мобильные приложения также используют гаммаволны и другие механизмы для непосредственного воздействия на мозговую активность пользователя. Эти приложения часто требуют более продолжительного и регулярного использования, чтобы достичь определенного результата, но они также могут существенно повлиять на работоспособность и концентрацию внимания.

Также важным элементом является удобство использования мобильного приложения. Чем проще и понятнее приложение, тем более вероятно, что пользователь будет использовать его регулярно. Однако, необходимо учитывать, что каждый пользователь индивидуален, и то, что работает для одного, может не работать для другого.

В целом, можно сказать, что создание мобильных приложений для улучшения концентрации является интересной и перспективной темой, которая может привести к улучшению качества жизни и работоспособности пользователей. Однако, необходимо учитывать особенности каждого пользователя и создавать удобные и эффективные инструменты для повышения концентрации внимания.

					ТПЖА.09.02.07.695577 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Игры могут быть отличным инструментом для развития и улучшения навыков концентрации внимания. Существует множество игр на различных платформах, которые могут помочь пользователям улучшить свои когнитивные способности и сосредоточенность, например:

1) Шахматы - игра, которая требует большой концентрации внимания и планирования. Она может помочь развить у пользователя навыки стратегического мышления и принятия решений.

2) Sudoku - игра-головоломка, которая требует логического мышления и способствует повышению концентрации внимания. Решение сложных задач, как в случае с судоку, может быть отличным способом для тренировки мозговых возможностей и улучшения внимательности элементарных задач.

3) Пасьянсы - игры, которые помогают тренировать память, логическое мышление и улучшают концентрацию внимания.

4) Игры, требующие управления фокусировкой - некоторые игры обучают пользователя контролировать свою концентрацию внимания. Например, приложение "Forest" блокирует пользовательские действия, если пользователь выходит из приложения, что помогает избежать отвлекающих факторов.

5) Игры с таймером - этот тип игр помогает тренировать пользователя, чтобы он мог сфокусироваться на решении задач в условиях ограниченного времени. Это требуется, например, в играх типа Tetris или решении кроссвордов.

Игры могут быть разнообразными, но их общей особенностью является то, что они затрагивают мозг все сложнее и сложнее с каждым уровнем, что благотворно сказывается на концентрации внимания. Кроме того, многие из этих игр доступны на мобильных платформах, что позволяет пользователям обучаться и тренироваться в любое время и в любом месте.

1.2 Обзор существующих аналогов

Существует множество мобильных приложений, которые могут помочь тренировать и улучшать концентрацию внимания. В качестве аналогов рассмотрим несколько примеров:

1) Headspace — это приложение с медитацией и контролем дыхания, которое может способствовать улучшению концентрации внимания. Headspace включает разные программы на 10, 15 или 20 минут, которые помогают заглушить шум вокруг и помочь пользователю сфокусироваться.

К положительным качествам приложения можно отнести следующее:

1) Занятия поделены на курсы с разными направлениями:

- Основа. Включает в себя 3 базовых курса, каждый из которых рассчитан на 10 дней. Здесь вы узнаете, как правильно медитировать, и, таким образом, плавно и неспешно войдете в мир медитативных практик;

- Здоровье: как избавиться от чувства тревоги, депрессии, снять стресс, избавиться от бессонницы, научит справляться с болью;

- Храбрость: как справляться с сомнениями и злостью на себя и окружающих, как принимать спонтанные и неприятные изменения;

- Счастье: как сосредоточиться на радости в жизни, дружелюбном отношении, трезвой самооценке и принятии;

- Работа: как концентрировать внимание на важном, ставить цели, применять творческий аспект в работе и соблюдать баланс между трудовой активностью и временем отдыха;

- Студенты: как справляться со стрессом, связанным с переездом в общежитие, как фокусироваться на получении новых знаний;

- Профессиональная сборка. Для глубокого погружения в себя;

- Разнообразные советы по мере прохождения курсов, например, медитация для концентрации внимания вам предложит несколько раз в день, не отвлекаясь, рассматривать любой предмет от 10 до 30 секунд.

					ТПЖА.09.02.07.695577 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

Из явных минусов можно выделить отсутствие русского языка и из бесплатного лишь курс новичка, за все остальные пакеты необходимо заплатить.

На рисунке 1 представлено несколько изображений из приложения.

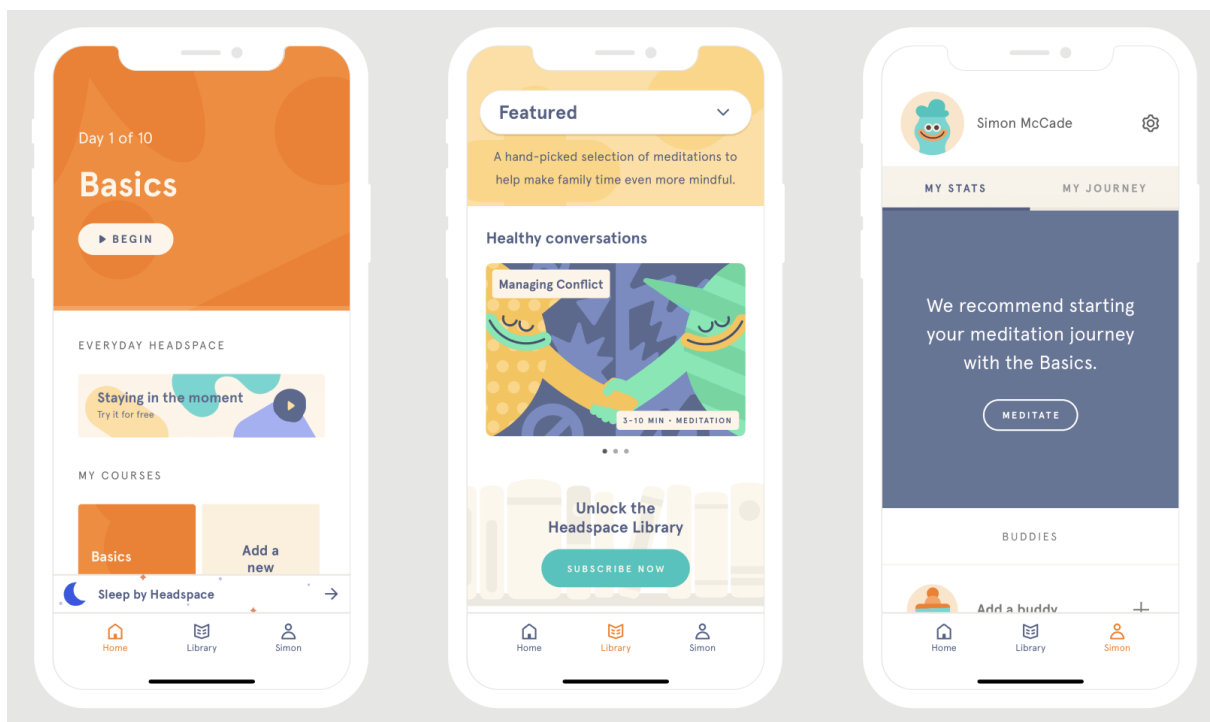


Рисунок 1 – Приложение Headspace

2) Lumosity — это приложение с коллекцией мини-игр, которые развивают когнитивные способности, такие как память, скорость реакции и концентрация внимания. Приложение Lumosity известно тем, что сочетает различные типы упражнений, которые помогают усовершенствовать несколько функций мозга одновременно.

Из положительных качеств приложения можно выделить следующее:

1) Все упражнения представлены в виде игр, что может привлечь больше детей и взрослых, которые хотели бы получить больше положительных эмоций.

2) Так же есть возможность играть бесплатно и без рекламы.

Из минусов можно выделить только отсутствие русского языка.

Приложение Lumosity в App Store представлено на рисунке 2



Lumosity – тренировка мозга 4+

Lumos Labs, Inc.

★★★★★ 4,6 • Оценок: 1,2 тыс.

Бесплатно • Включает встроенные покупки

Снимки экрана [iPhone](#) [iPad](#)

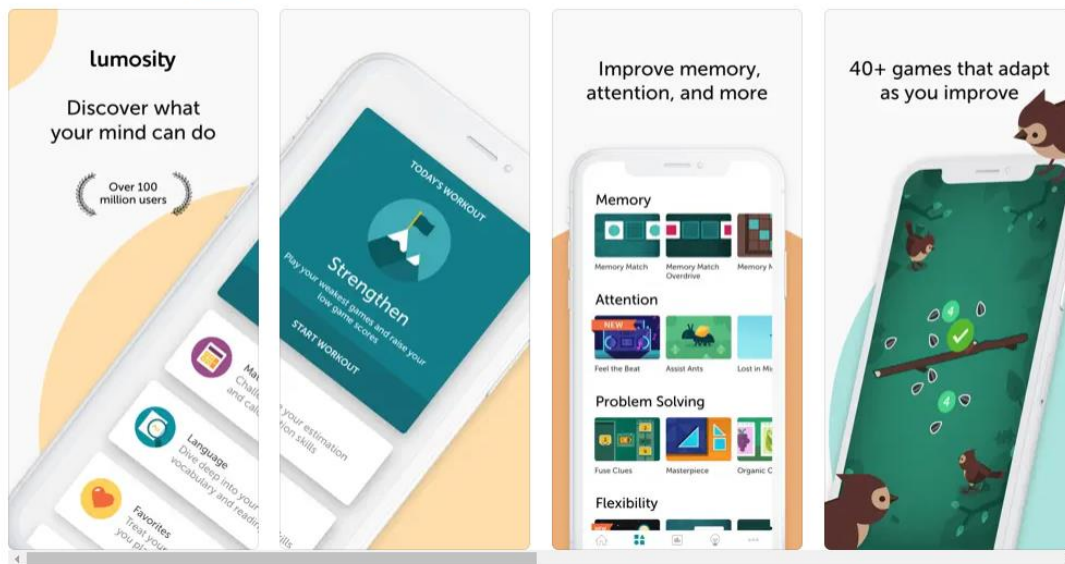


Рисунок 2 – Приложение Lumosity в App Store

3) Focus@Will — это приложение, уникальной фицей которого является музыкальная подборка, которая способствует концентрации и эффективности в работе. Музыка, которую предоставляет Focus@Will, подобрана на основе исследований действия звуковых вибраций на мозг и позволяет работать на полную мощность.

К положительным качествам можно отнести следующее:

1) Приложение позволяет своим пользователям персонализировать эффективность улучшения фокусировки, пропуская треки, которые они считают отвлекающими;

2) Плеер работает на HTML5, что означает, что вы можете использовать его практически на любом устройстве.

Изм.	Лист	№ докум.	Подпись	Дата

ТПЖА.09.02.07.695577 ПЗ

Лист

9

Из минусов:

- 1) Отсутствие выбора исполнителя;
- 2) Вся музыка использует инструменты и в ней нету вокала;
- 3) Русский язык отсутствует;
- 4) Только месяц бесплатного периода, после указанного срока сервис попросит заплатить за подписку.

Внешний вид сервиса представлен на рисунке 3

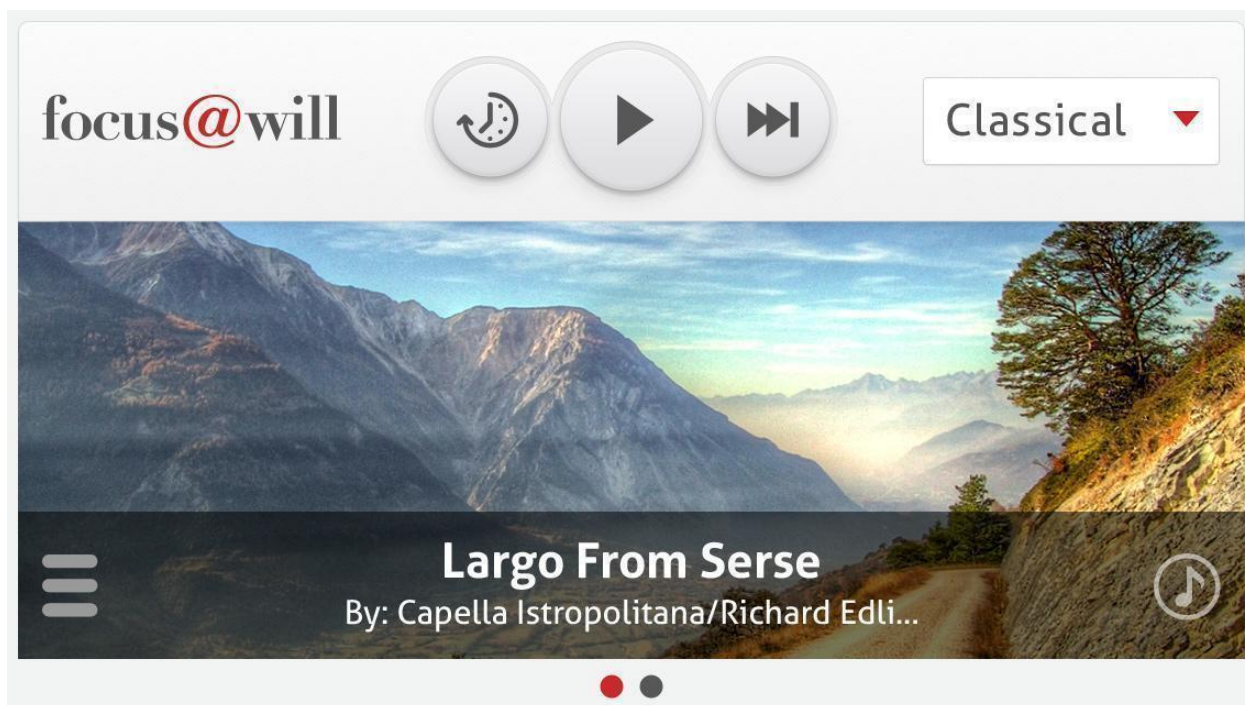


Рисунок 3 – Сервис Focus@Will

4) Peak — это приложение с коллекцией упражнений, которые тренируют мозг в области концентрации внимания, памяти и других когнитивных навыков. Peak включает программу обучения, которая помогает пользователям развивать свои способности и максимизировать потенциал.

Плюсы приложения:

- 1) Собирает статистику, благодаря которой можно выявить свои слабые и сильные стороны;

- 2) Можно поставить напоминания, чтобы тренироваться регулярно;
- 3) Процесс тренировок представлен в виде игр.

Минусы:

- 4) Более 90% упражнений платные
- 5) Отсутствие русского языка

Превью приложения из App Store представлено на рисунке 4

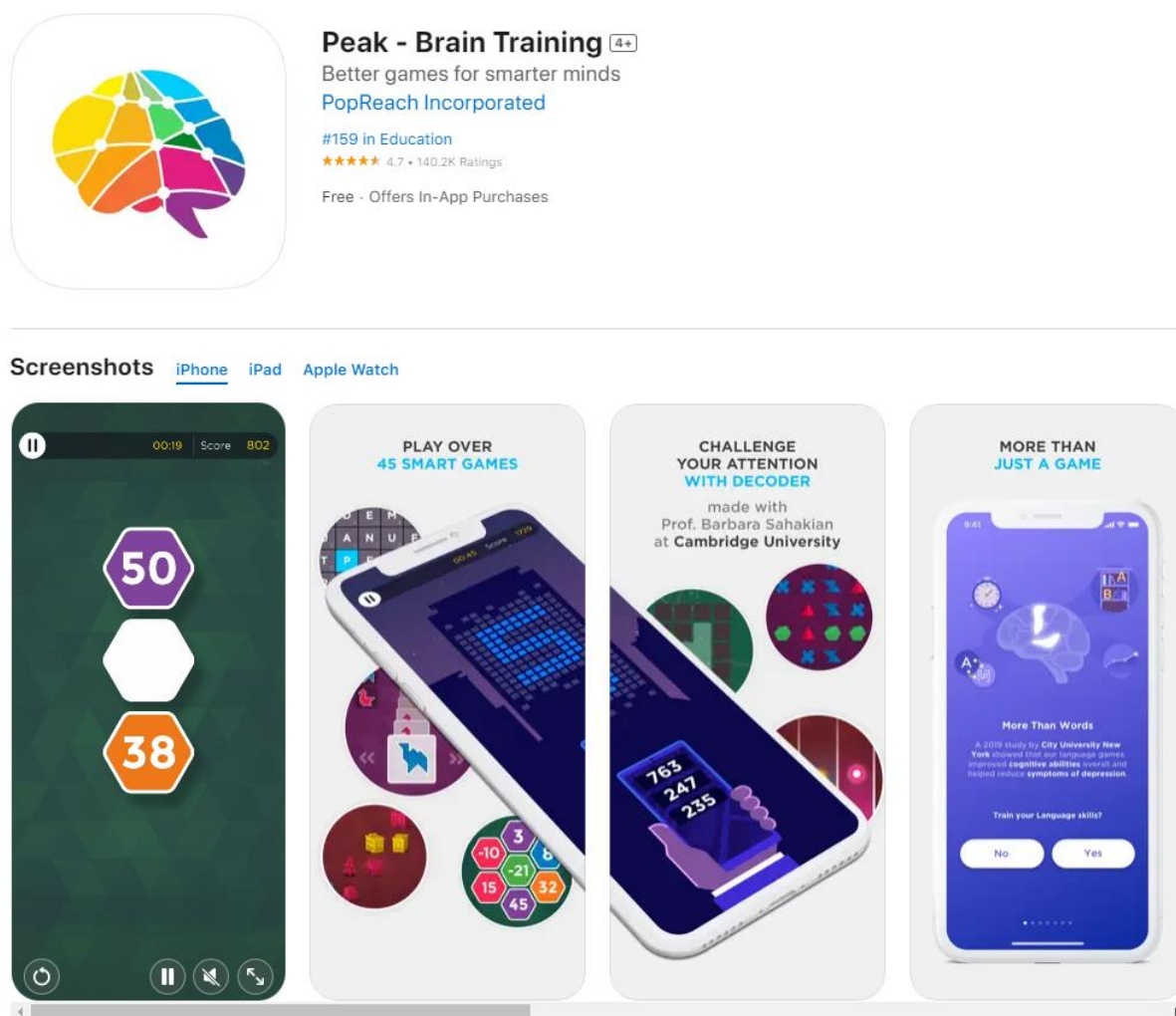


Рисунок 4 – Превью приложения Peak

5) Brain.fm — это приложение, похожее на Focus@Will, которое предоставляет музыку, настроенную на частоты, помогающие улучшить концентрацию внимания. Brain.fm использует научный подход для создания музыки, которая может помочь пользователю сосредоточиться, бросить вызов отвлекающим факторам и повысить продуктивность.

Плюсы приложения:

- 1) Различные таймеры, направленные на решения разных задач;
- 2) Предлагает помощь в медитации, улучшении сна и концентрацию;
- 3) Сервис доступен для любых устройств.

4) Минусы:

- 5) Отсутствие русского языка;
- 6) Бесплатный период только 3 дня, дальше необходимо оформлять платную подписку.

Внешний вид приложения представлен на рисунке 5.

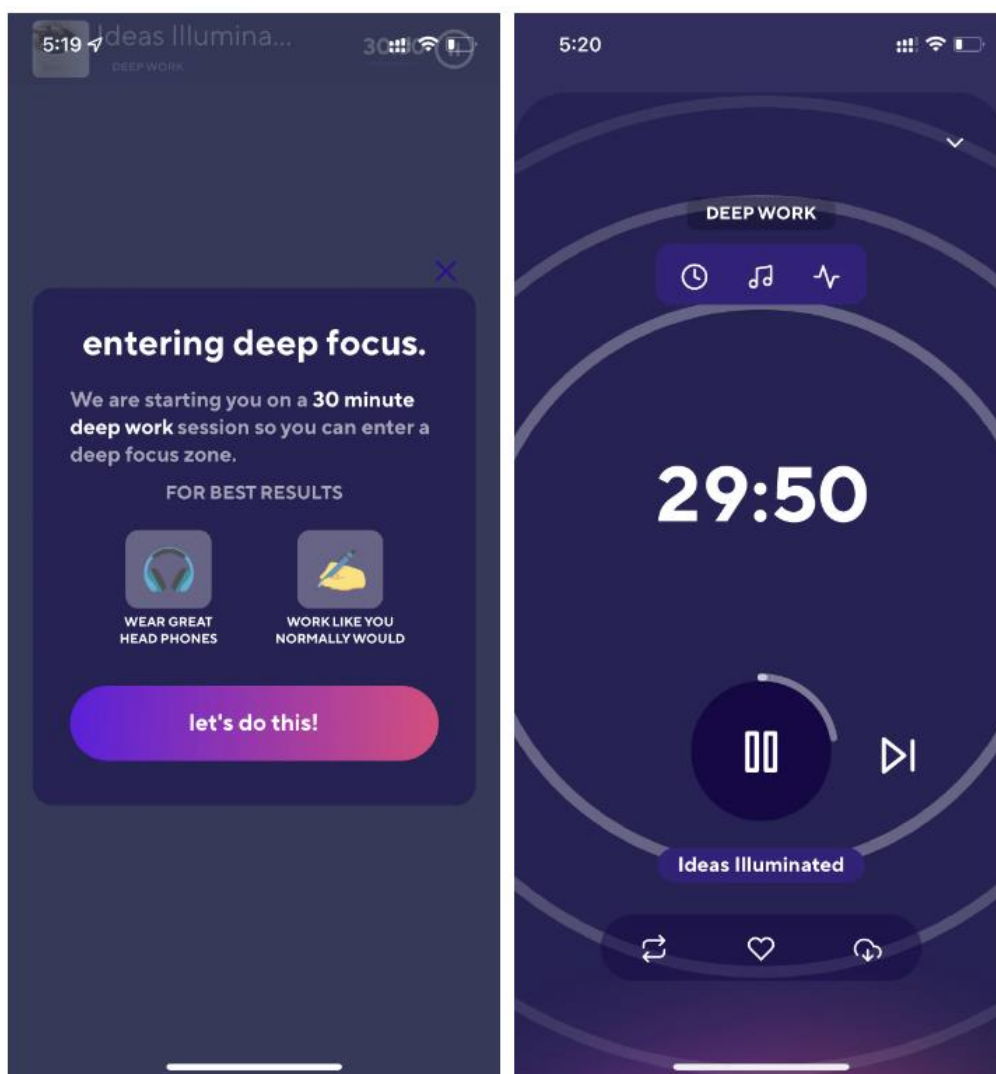


Рисунок 5 – Приложение Brain.fm

Эти приложения могут быть полезны для развития и улучшения навыков концентрации внимания. Они доступны на iOS и Android — это значит, что пользователи могут использовать их в любом месте и в любое время.

В результате обзора аналогов была создана сводная таблица (Таблица 1).

Таблица 1 – Сводная таблица аналогов

Наименование	Просмотр статистики	Возможность выбора игр / музыки	Русский язык	Присутствие бесплатного, неограниченного функционала
Headspace	+	+	-	+
Lumosity	+	+	-	+
Focus@Will	+	-	-	-
Peak	+	+	-	+
Brain.fm	+	-	-	-
Разрабатываемое приложение	+	+	+	+

Главной проблемой представленных аналогов является отсутствие русского языка и большая часть функционала приложений либо платная, либо бесплатная на ограниченный срок, данные проблемы будут решены в разрабатываемом приложении.

1.3 Анализ проблемной области

В современном мире внимание — это ценный ресурс, который часто игнорируется. Сегодня мы живем в мире контент-маркетинга и социальных сетей, где различные компании и бренды борются за наше внимание. В результате мы перегружены информацией, и наше внимание становится ограниченным ресурсом.

Каждый день мы сталкиваемся с десятками или даже сотнями различных предложений и получаем разную информацию одновременно. Но наш мозг не может обрабатывать все эти данные, что приводит к проблемам с памятью, сосредоточенностью и активности.

Чтобы управлять своим вниманием и использовать его эффективно, нужно научиться выбирать, какую информацию мы хотим получать. Мы должны осознавать, что наше внимание — это ценный ресурс, и мы должны управлять им так же, как управляем нашим временем и ресурсами. Нужно убедиться, что мы используем свое внимание для тех вещей, которые действительно важны для нас, и которые помогают нам достигать наших целей.

Не менее важной проблемой являются социальные сети и наше внимание к ним. Проблема состоит в том, что многие люди проводят слишком много времени, листая ленты новостей и интерактивные карусели мобильных приложений. Это приводит к тому, что человек не может сосредоточиться и успешно выполнить задачи, а также уменьшает его способность к общению в реальной жизни.

Мобильное приложение для развития концентрации внимания может решить ряд проблем, связанных с самим вниманием. Вот несколько примеров:

1) Улучшение работоспособности: когда человек не может достаточно сосредоточиться на работе, производительность снижается. Мобильное приложение может помочь увеличить концентрацию внимания, и тем самым увеличить работоспособность.

2) Улучшение учебной деятельности: студентам, которые сталкиваются с проблемами концентрации, может помочь мобильное приложение для развития концентрации. Это может улучшить их успеваемость и повысить мотивацию к учебе.

3) Улучшение психического здоровья: недостаток концентрации может привести к беспокойству, депрессии и другим психическим заболеваниям. Мобильное приложение может помочь снять стресс, повысить настроение и снизить риск зависимости от алкоголя и других вредных привычек.

4) Улучшение качества жизни: недостаток концентрации может привести к различным проблемам в повседневной жизни, например, пропущенным срокам, забытым встречам и не выполненным задачам. Мобильное приложение может помочь улучшить качество жизни и снизить стресс, связанный с этими проблемами.

Таким образом, мобильное приложение для развития концентрации внимания может иметь положительный эффект на все аспекты жизни, связанные с недостатком концентрации внимания.

Выводы по разделу.

В качестве выводов по данному разделу могу сказать следующее, что в рамках данного раздела был проведен анализ предметной области, в которой было рассмотрено для чего необходима концентрация внимания и как приложения по ее развитию может оказать положительное влияния на пользователя, после чего были рассмотрены уже существующие аналоги мобильных или Web приложений, их описание, недостатки и положительные качества. Затем был рассмотрен анализ предметной области, в которой подробно была расписана проблема недостатка внимания и как эту проблему может решать приложение по ее развитию.

2 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

2.1 Архитектура системы

Архитектура клиент-серверного приложения [1] — это модель, в которой клиент (приложение) отправляет запросы на сервер, а сервер обрабатывает запросы и отправляет ответы. Два компонента архитектуры, клиент и сервер, взаимодействуют друг с другом с помощью сетевого протокола.

Как правило, все данные необходимые для хранения и использования в приложении, находится на стороне веб-сервера. В этом положении сервер управляет входящими запросами на основе логики продукта и формирует ответы, которые должны быть отправлены пользователю. В это же время клиент получает ответ и обрабатывает его в соответствии с его логикой.

Существует несколько типов клиент-серверной архитектуры, в том числе следующие:

1) Двухуровневая архитектура. Эта архитектура также известна как модель клиент-сервер, в которой клиент взаимодействует напрямую с сервером для получения информации. В этой модели клиент отвечает за работу с пользовательским интерфейсом, а сервер занимается хранением и обработкой данных. Диаграмма двухуровневой архитектуры представлена на рисунке 6.

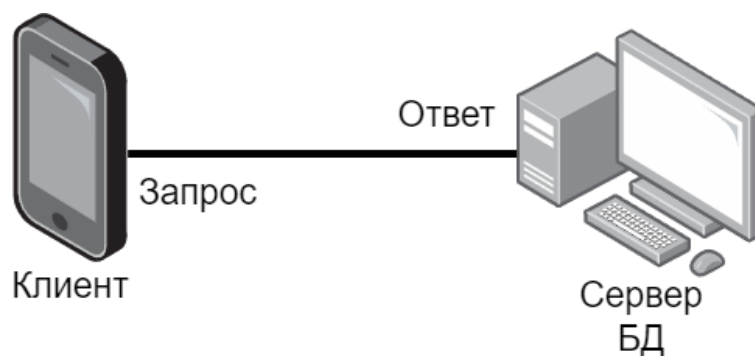


Рисунок 6 – Диаграмма двухуровневой архитектуры клиент-сервер

2) Трехуровневая архитектура. Эта архитектура имеет дополнительный уровень между клиентом и сервером, известный как уровень приложения или промежуточного программного обеспечения. Сервер промежуточного программного обеспечения управляет связью между клиентом и сервером базы данных. Эта архитектура обеспечивает лучшую масштабируемость, производительность и безопасность. Трехуровневая архитектура представлена на рисунке 7.

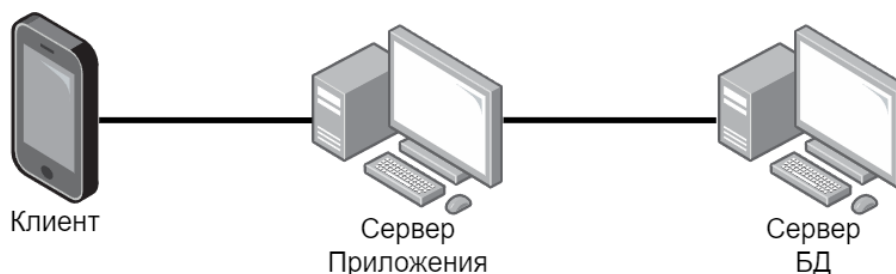


Рисунок 7 – Диаграмма трехуровневой архитектуры клиент-сервер

3) Многоуровневая архитектура. Эта архитектура имеет несколько уровней или уровней, которые существуют между клиентом и сервером. Количество уровней зависит от сложности приложения и требуемых услуг. Эта архитектура особенно полезна при работе с крупномасштабными системами, поскольку она обеспечивает высокую масштабируемость, отказоустойчивость и производительность.

4) Одноранговая архитектура. В этой архитектуре нет различий между клиентом и сервером. Все узлы в сети равны, и каждый узел может запрашивать и предоставлять услуги другим узлам в сети. Эта архитектура обычно используется в приложениях для обмена файлами и децентрализованных системах.

В целом выбор правильного типа архитектуры клиент-сервер зависит от конкретных потребностей приложения, включая требования к производительности, масштабируемости и безопасности.

В нашем случае лучшим решением может стать двухуровневая архитектура, в которой безопасность данных от внешнего воздействия будет обеспечена. Более крупные архитектуры лишь усложнят процесс разработки, поэтому они не нужны для системы.

Диаграмма связи клиента и сервера представлена на рисунке 8.

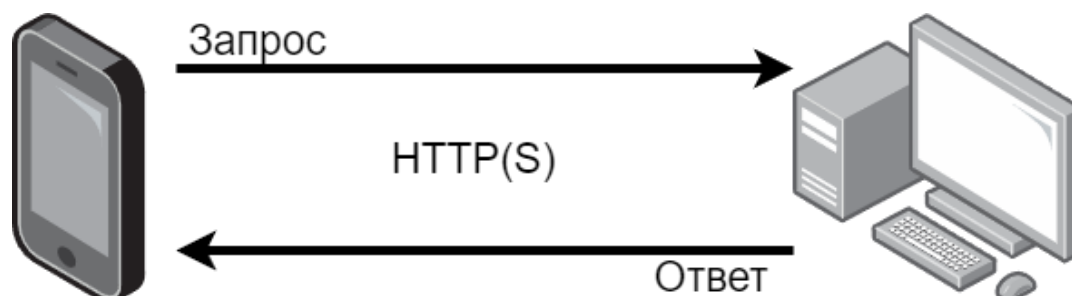


Рисунок 8 – Диаграмма связи клиента и сервера

Связь между клиентом и сервером основана на протоколе HTTP [3]. Сервер прослушивает запросы, поступающие от клиента на определенный номер порта, и отправляет ответы клиенту на тот же порт, диаграмма связи представлена на рисунке 8.

2.2 Проектирование структуры ПО

Структура приложения на React Native [4], аналогична структуре типичного приложения React. Компоненты в React Native состоят из комбинации собственных компонентов и кода JavaScript. Собственные компоненты — это элементы пользовательского интерфейса, отображаемые непосредственно операционной системой, такие как ввод текста, кнопки и изображения. Код JavaScript используется для определения поведения и взаимодействия этих компонентов.

Точкой входа в приложение React Native обычно является файл App.js, который действует как отправная точка для приложения и отвечает за отрисовку первого компонента. Оттуда компоненты организованы в папки в зависимости от их функциональности или функции и импортируются по мере необходимости.

В целом структура приложения React Native аналогична типичному приложению React, но с дополнительными соображениями по поводу нативных компонентов и интеграции с мобильной операционной системой.

Для просмотра результата проделанной работы и просмотра ошибок от каждой из частей приложения можно использовать инструмент Expo.

Expo — это платформа с открытым исходным кодом для создания мобильных приложений. Он предоставляет множество полезных инструментов и сервисов, упрощающих разработчикам создание высококачественных мобильных приложений. Одной из ключевых особенностей Expo является встроенная поддержка многих популярных сторонних библиотек и API, включая Google Maps, аутентификацию Facebook и push-уведомления. Это означает, что разработчики могут быстро и легко добавлять эти функции в свои приложения без необходимости написания собственного кода.

Кроме того, Expo предоставляет мощный набор инструментов разработки, включая интерфейс командной строки (CLI) и сервер разработки. CLI позволяет разработчикам быстро создавать новые проекты, запускать тесты и публиковать свои приложения в магазинах приложений. Сервер разработки позволяет разработчикам быстро просматривать изменения в своих приложениях в режиме реального времени. Данный инструмент может оказаться наиболее комфортным для решения будущих проблем с написанием кода, поэтому следует выбрать именно его.

Структуру приложения на React Native можно описать так: Используя Ехро CLI создаем новый проект с базовой структурой и зависимостями. Основной файл нашего приложения — “App.js”. Этот файл содержит корневой компонент приложения и отвечает за отрисовку всех остальных компонентов. Через Ехро CLI запускаем проект и сервер передает все данные приложения на любое устройство имеющее подключение к одной сети и приложение Ехро Go, сканируем QR – код и наше приложение отображается на экране мобильного устройства.

Для большего удобства была спроектирована примерная файловая структура приложения, представленная на рисунке 9.

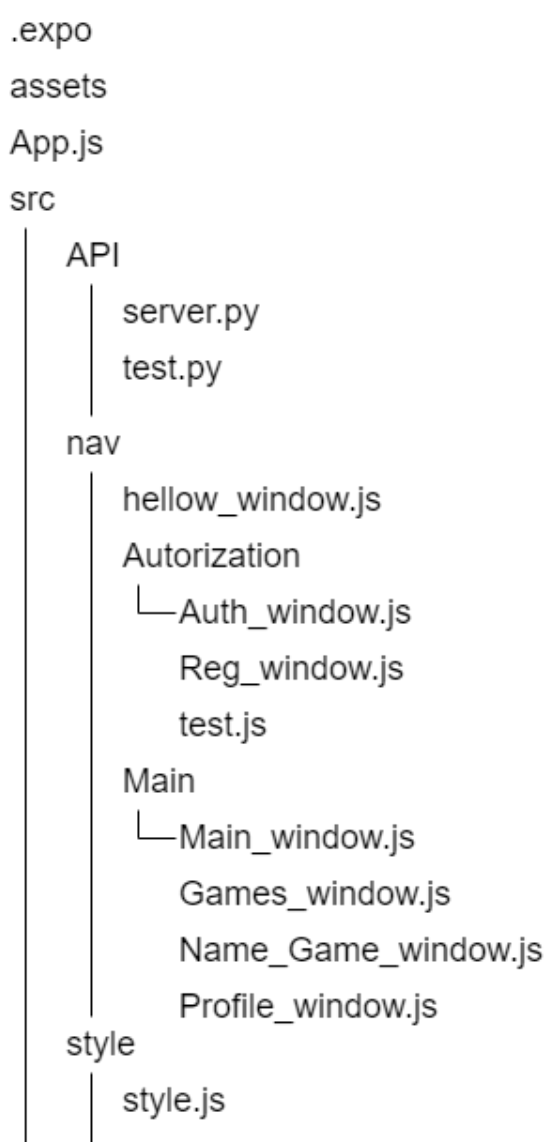


Рисунок 9 – Файловая структура приложения

Папка API на конечном этапе разработки будет перемещена в отдельное пространство для защиты от возможного проникновения внешними пользователями, но на данный момент для удобства разработки будет находиться вместе с проектом приложения.

2.3 Проектирование структуры сервера

Сервер — это компьютер или устройство, которое предоставляет ресурсы или услуги другим компьютерам или устройствам, подключенным к сети.

Существует много типов серверов, в том числе:

1) Веб-серверы. Эти серверы используются для размещения веб-сайтов и веб-приложений.

2) Почтовые серверы. Эти серверы управляют обменом электронной почтой и обеспечивают доставку сообщений нужным получателям.

3) Файловые серверы. Эти серверы хранят файлы и обмениваются ими по сети, что позволяет пользователям получать доступ к файлам с разных устройств.

4) Серверы баз данных. Эти серверы управляют базами данных и предоставляют доступ к ним приложениям и пользователям.

5) Серверы печати. Эти серверы управляют службами печати и позволяют нескольким пользователям печатать на одном и том же принтере.

В нашем случае лучшим решением подойдет сервер базы данных, поскольку нам необходимо хранить данные и использовать их для клиента.

Чтобы более подробно разобраться в структуре сервера были сделаны схемы, расположенные на рисунках 10 и 11.

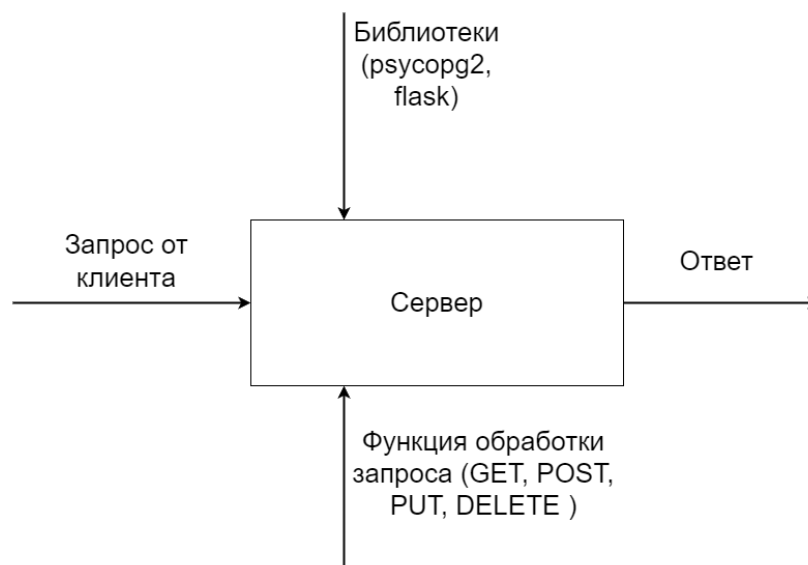


Рисунок 10 – Схема IDEF 0 сервера

Сервер получает запрос от клиента в качестве входных данных по маршруту, заданному на сервере и клиенте, маршрутом является URL – адрес, переходя по которому сервер запускает функцию. Функция же обрабатывает входные данные, если такие имеются, после чего устанавливает связь с базой данных или выполняет другие операции, а затем возвращает ответ пользователю, который обычно представляет собой HTML [5] страницу или данные JSON. Flask и psycopg 2 – библиотеки являющиеся контроллерами для функций.

Flask - это микро веб-фреймворк [6], написанный на Python. Он разработан, чтобы быть легким и модульным, что позволяет разработчикам быстро и легко создавать веб-приложения. Идеальный вариант в качестве инструмента для создания простого сервера в короткие сроки.

Psycopg 2 — это адаптер базы данных PostgreSQL для языка программирования Python. Это популярный выбор для разработчиков Python, которым необходимо взаимодействовать с базами данных PostgreSQL из-за его скорости, многопоточности, простоты и надежности, поэтому он нам подойдет.

Для более подробного представления работы функций на сервере схема IDEF 0 на рисунке 10 была декомпозирована и в качестве примера представлена на рисунке 11. На схеме рассматривается случай, когда клиент отправляет POST запрос на добавление нового пользователя в базу данных.

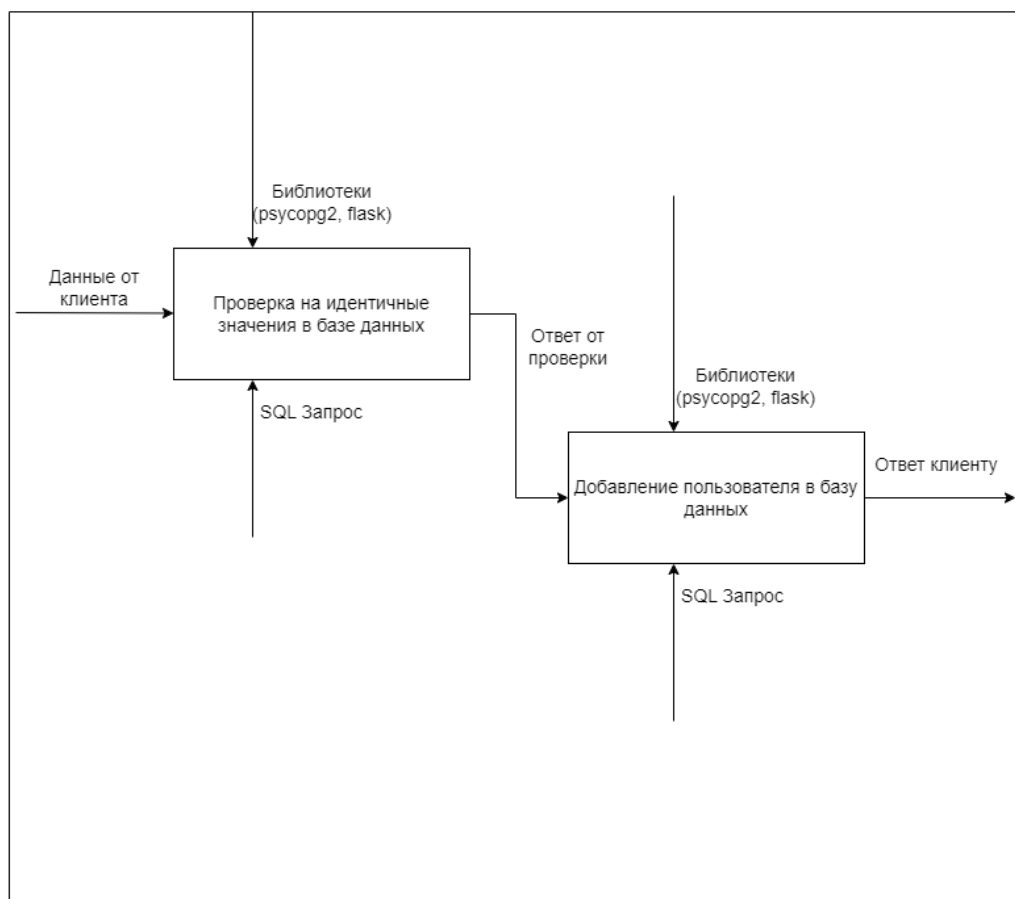


Рисунок 11 – Схема декомпозиции IDEF 0

Полученные сервером данные отправляются на проверку в функцию в которой идет обращение к базе данных через SQL запрос на идентичность данных, после чего, если пользователей идентичных данным от клиента нет, то следует добавление пользователя в базу данных, так же через SQL запрос. Затем пользователю отправляется ответ об успешной обработке данных.

2.4 Проектирование интерфейса

Хорошо спроектированный интерфейс может сделать продукт простым в использовании, улучшить его функциональность и помочь во взаимодействии с пользователем. Это помогает пользователям более эффективно перемещаться и взаимодействовать с продуктом или приложением, делая его более приятным и удобным для пользователя.

Более того, хороший дизайн интерфейса также может повысить вовлеченность, привлечь больше пользователей и побудить их вернуться к продукту. Другими словами, дизайн интерфейса играет решающую роль в определении успеха любого цифрового продукта или приложения.

Именно по этим причинам необходимо проектирование интерфейса, а в следствии и необходимы инструменты для ее создания. Таким инструментом может стать интернет-ресурс Figma.

Figma — это облачное программное обеспечение для проектирования, которое позволяет дизайнерам и командам совместно работать над проектами в режиме реального времени. Figma также предлагает широкий спектр компонентов и шаблонов дизайна, которые можно настроить для конкретных проектов. Эти компоненты и шаблоны предназначены для многократного использования, что помогает ускорить процесс проектирования и обеспечивает согласованность между проектами. Помимо возможностей дизайна, Figma также предлагает мощный инструмент прототипирования, который позволяет дизайнерам создавать интерактивные прототипы своих проектов.

В целом Figma — это надежный и удобный инструмент для проектирования его компоненты дизайна и возможности прототипирования делают его мощным инструментом для создания цифровых проектов.

В ходе проектирования интерфейса программы было сделано несколько форм, для наглядного представления будущ его интерфейса программы, данные формы представлены на рисунках 12, 13 и 14.

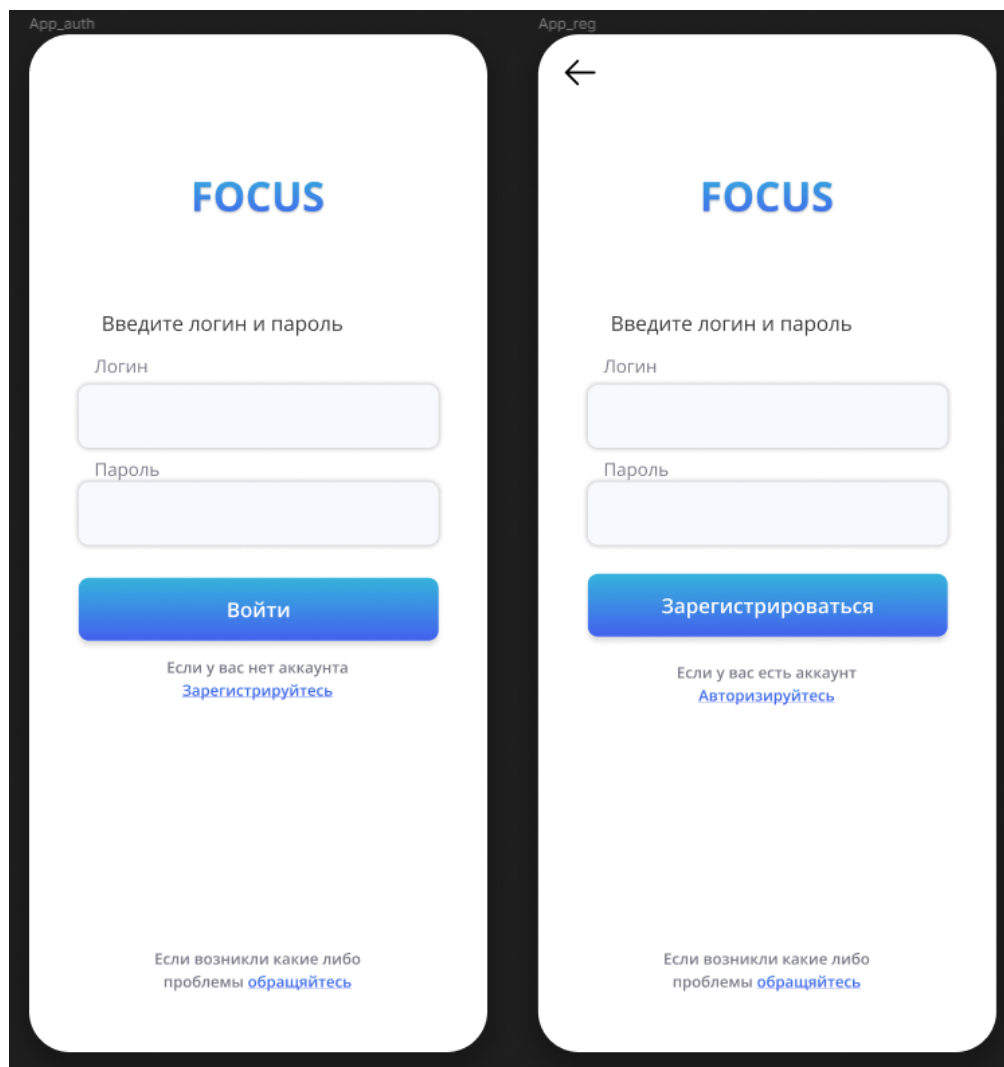


Рисунок 12 – Макеты страниц авторизации и регистрации



Рисунок 13 – Макет главной страницы

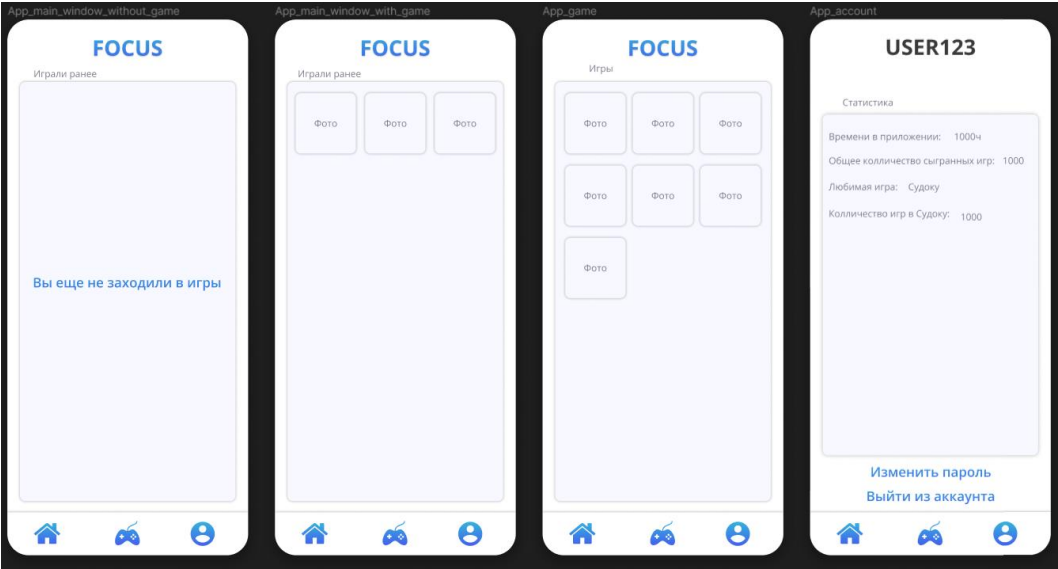


Рисунок 14 – Макеты страниц приложения после авторизации

2.5 Разработка алгоритмов функционирования ПО

Для обеспечения того, чтобы программное обеспечение функционировало должным образом и достигало своей цели, лучшим решением может стать разработка алгоритмов функционирования программного обеспечения [7]. В его разработку в основном входят различные процессы, такие как анализ требований, разработка блок-схем и диаграмм, кодирование алгоритмов на языке программирования, тестирование и отладка кода.

Блок-схема — это особый тип диаграммы, которая представляет компоненты системы или процесса в иерархической структуре. Он использует блоки для представления отдельных компонентов или подсистем, а также линии или стрелки для отображения связей или взаимодействий между ними.

Для более подробного представления работы приложения была спроектирована схема алгоритма работы приложения, изображенная на рисунке 15.

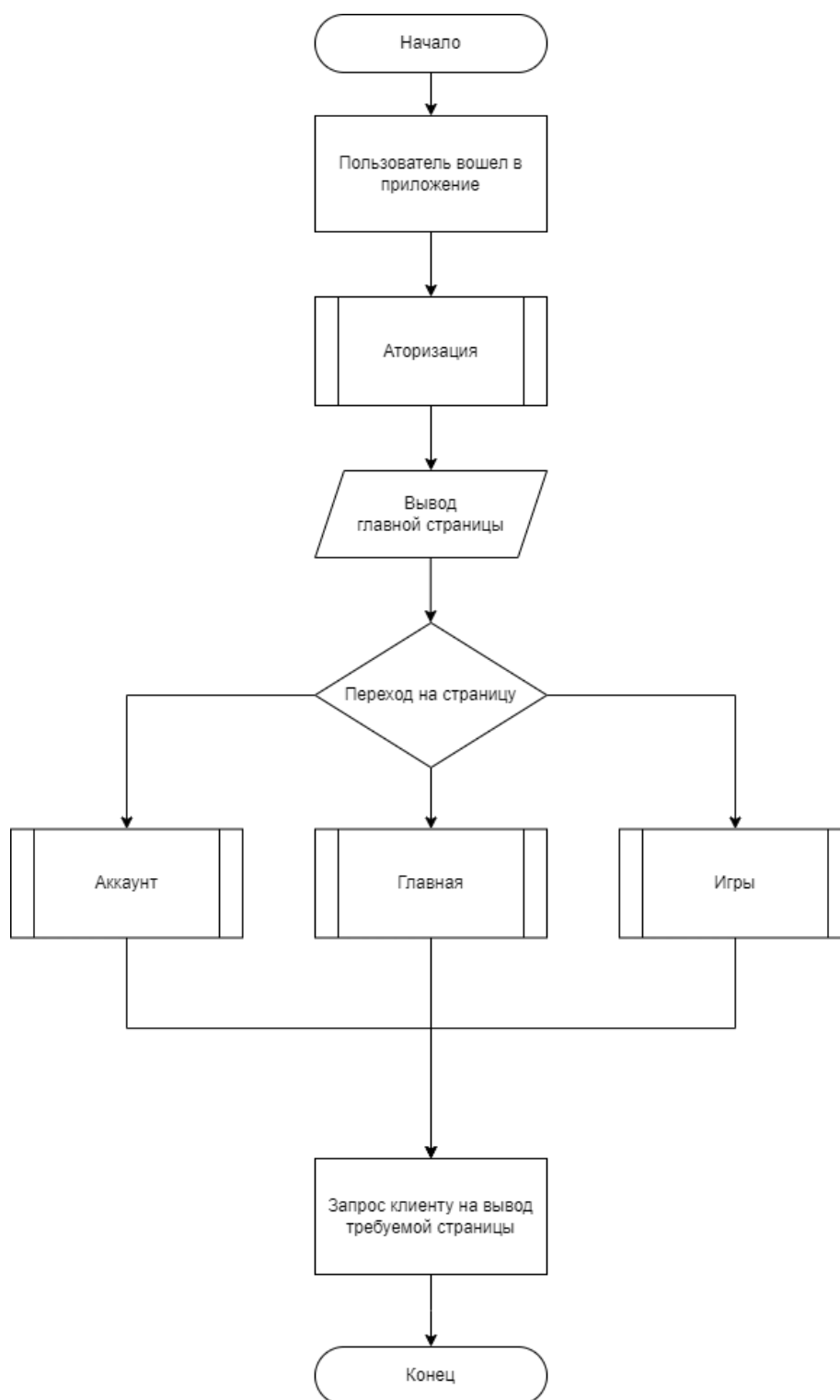


Рисунок 15 – Схема алгоритма работы приложения

Для понимания процесса работы авторизации, была так же спроектирована схема использования алгоритма авторизации, изображенная на рисунке 16.

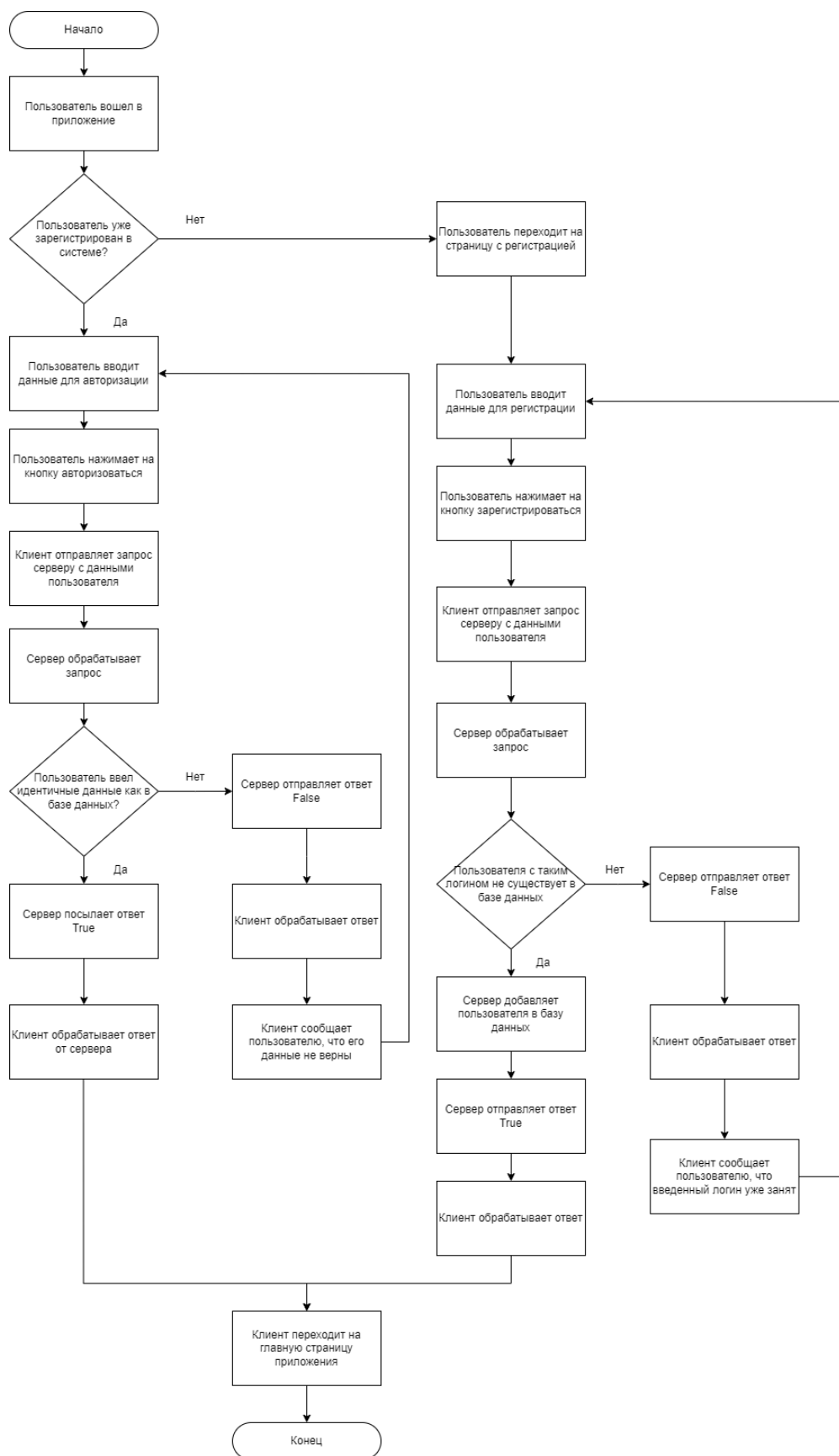


Рисунок 16 – Схема использования алгоритма авторизации

Использование схемы алгоритма авторизации может оказать крупную поддержку на стадии разработки приложения, так как подробно описывает какие алгоритмы и их последовательность необходима как для сервера, так и для клиента, поэтому она важна на стадии проектирования алгоритмов работы приложения.

В качестве выводов по данному разделу могу сказать следующее, что в рамках данного раздела было проведено проектирование архитектуры системы, в которой была рассмотрена модель архитектуры клиент-серверного приложения, ее типы и их описание с приведенными диаграммами, после чего было проведено проектирование структуры программного обеспечения, в которой описана структура приложения с использованием React Native, а так же спроектирована файловая структура приложения. Затем было проведено проектирование серверной части приложения, в котором описано о серверах, какие они бывают и были спроектированы схемы, что более подробно показывают принцип работы сервера. Следующим этапом стало проектирование интерфейса, в котором описано для чего оно необходимо и приведены макеты страниц разрабатываемого приложения. И заключительным этапом стало разработка алгоритмов функционирования программного обеспечения, в котором описано для чего оно необходимо и описано какие схемы были спроектированы в рамках данного подраздела.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

3.1 Выбор и основания средств разработки

Для создания программного продукта были задействованы следующие языки программирования и технологии:

В качестве редактора кода был выбран Visual Studio Code — популярный редактор кода с открытым исходным кодом, разработанный Microsoft. Он доступен для операционных систем Windows, macOS и Linux. Он предлагает широкий спектр функций и расширений, которые делают его популярным среди разработчиков.

Некоторые из ключевых функций Visual Studio Code включают в себя:

1) IntelliSense: это интеллектуальный инструмент завершения кода, который помогает разработчикам писать код более эффективно, предлагая фрагменты кода, переменные и методы на основе контекста кода.

2) Отладчик: Visual Studio Code поставляется со встроенным отладчиком, который позволяет разработчикам отлаживать свой код прямо из редактора.

3) Интеграция с Git. Visual Studio Code легко интегрируется с Git, что позволяет разработчикам более эффективно управлять своими репозиториями кода.

4) Расширения: Visual Studio Code имеет обширный репозиторий расширений, которые разработчики могут использовать для улучшения своего опыта кодирования.

5) Live Share: это новая функция Visual Studio Code, которая позволяет разработчикам совместно работать над одной кодовой базой в режиме реального времени, независимо от их местонахождения.

В целом Visual Studio Code — отличный инструмент для разработчиков, которые хотят писать код быстро и эффективно. Его обширный набор функций и расширений делает его мощным редактором кода.

Интерфейс программы представлен на рисунке 17.

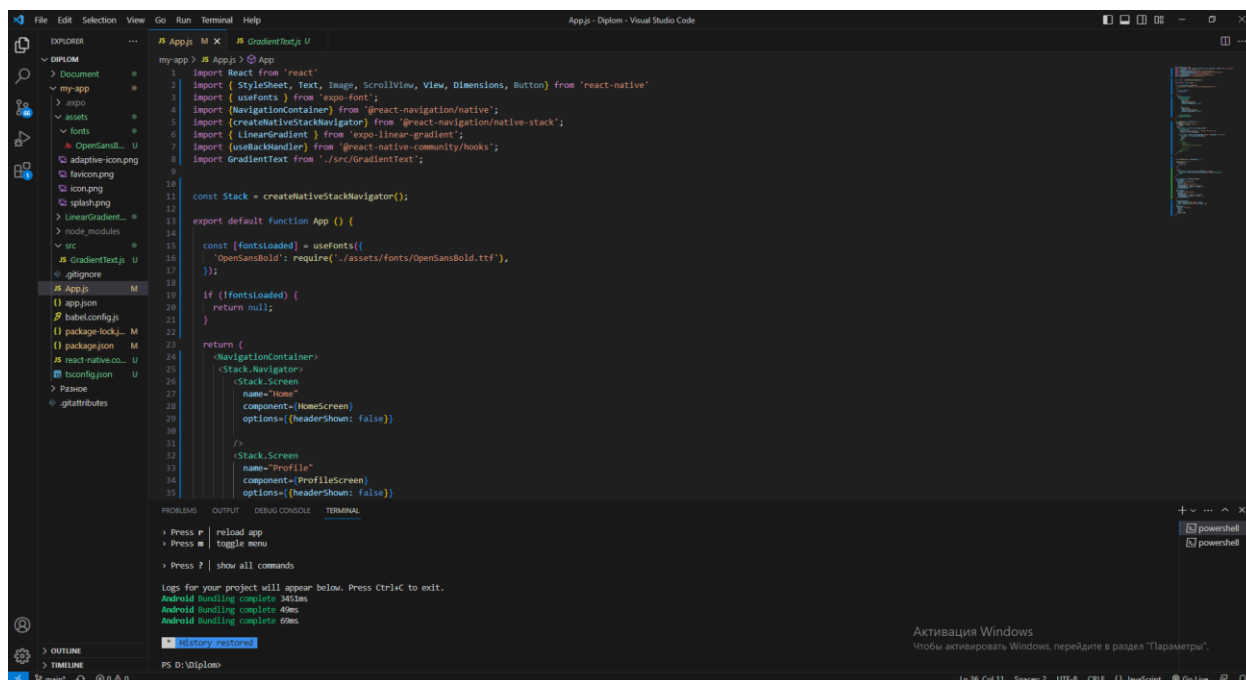


Рисунок 17 – Интерфейс в Visual Studio Code

В качестве языка программирования для написания интерфейса и его логики был выбран React Native — это платформа с открытым исходным кодом, разработанная Facebook для создания мобильных приложений с использованием JavaScript и React. Это позволяет разработчикам создавать высококачественные мобильные приложения для iOS и Android, используя единую кодовую базу. React Native предоставляет набор готовых компонентов, которые можно использовать для создания пользовательского интерфейса приложения.

Одним из ключевых преимуществ React Native является то, что он использует подход «обучись один раз, пиши, где угодно», что означает, что разработчики, знакомые с React, могут использовать одни и те же навыки для создания мобильных приложений для платформ iOS и Android. Это помогает сократить время и стоимость разработки для предприятий, которым необходимо создавать приложения для обеих платформ.

React Native также предлагает ряд оптимизаций производительности, таких как собственный рендеринг и асинхронное выполнение кода JavaScript, которые помогают повысить производительность и скорость отклика приложения. Кроме того, он предоставляет функцию горячей перезагрузки, которая позволяет разработчикам видеть внесенные ими изменения в режиме реального времени без необходимости перезапускать приложение.

Пример кода на React Native представлен на рисунке 18.

```
const Stack = createNativeStackNavigator();

export default function App () {

  const [fontsLoaded] = useFonts({
    'OpenSansBold': require('./assets/fonts/OpenSansBold.ttf'),
  });

  if (!fontsLoaded) {
    return null;
  }

  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{headerShown: false}}
        />
        <Stack.Screen
          name="Profile"
          component={ProfileScreen}
          options={{headerShown: false}}
        />
      </Stack.Navigator>
    </NavigationContainer>
  )
}
```

Рисунок 18 – Пример кода на языке программирования React Native

Для хранения данных и их безопасности на стадии проектирования было решено сделать архитектуру клиент-сервер, в которой обязательным звеном является сам сервер, и его создание было решено проводить с помощью языка программирования python. Почему именно Python? Python — это высокоуровневый язык программирования общего назначения. Есть несколько причин, по которым Python является отличным языком для программирования на стороне сервера:

1) Простота изучения и использования. Python — очень удобный язык с простым для понимания синтаксисом. Это язык высокого уровня, что означает, что он абстрагируется от многих низкоуровневых деталей программирования, что делает его доступным для начинающих.

2) Масштабируемость: Python обладает высокой масштабируемостью, что означает, что он может без проблем обрабатывать большие объемы данных и трафика.

3) Большое и активное сообщество: Python имеет большое и активное сообщество разработчиков, которые вносят свой вклад в его развитие и предлагают поддержку другим разработчикам. Это означает, что в Интернете доступно множество ресурсов, таких как библиотеки и фреймворки, которые могут упростить и повысить эффективность программирования на стороне сервера.

4) Безопасность: Python имеет встроенные функции безопасности, которые делают его очень безопасным языком для программирования на стороне сервера. Например, он имеет безопасные хеш-функции и библиотеки для шифрования и дешифрования.

В заключение Python — отличный язык для создания серверов благодаря простоте использования, масштабируемости, большому сообществу и встроенным функциям безопасности.

Так как в качестве языка программирования был выбран python, то по сообразным решениям в качестве IDE для разработки на нем был выбран PyCharm. PyCharm — это мощная и популярная интегрированная среда разработки для языка программирования Python. Он также предоставляет удобный интерфейс для создания и управления базами данных, работы с системами контроля версий, такими как Git и GitHub, и интеграции с различными инструментами и серверами.

Для ознакомления с интерфейсом IDE PyCharm и кода на языке Python, можете просмотреть рисунок 19, на котором они представлены.

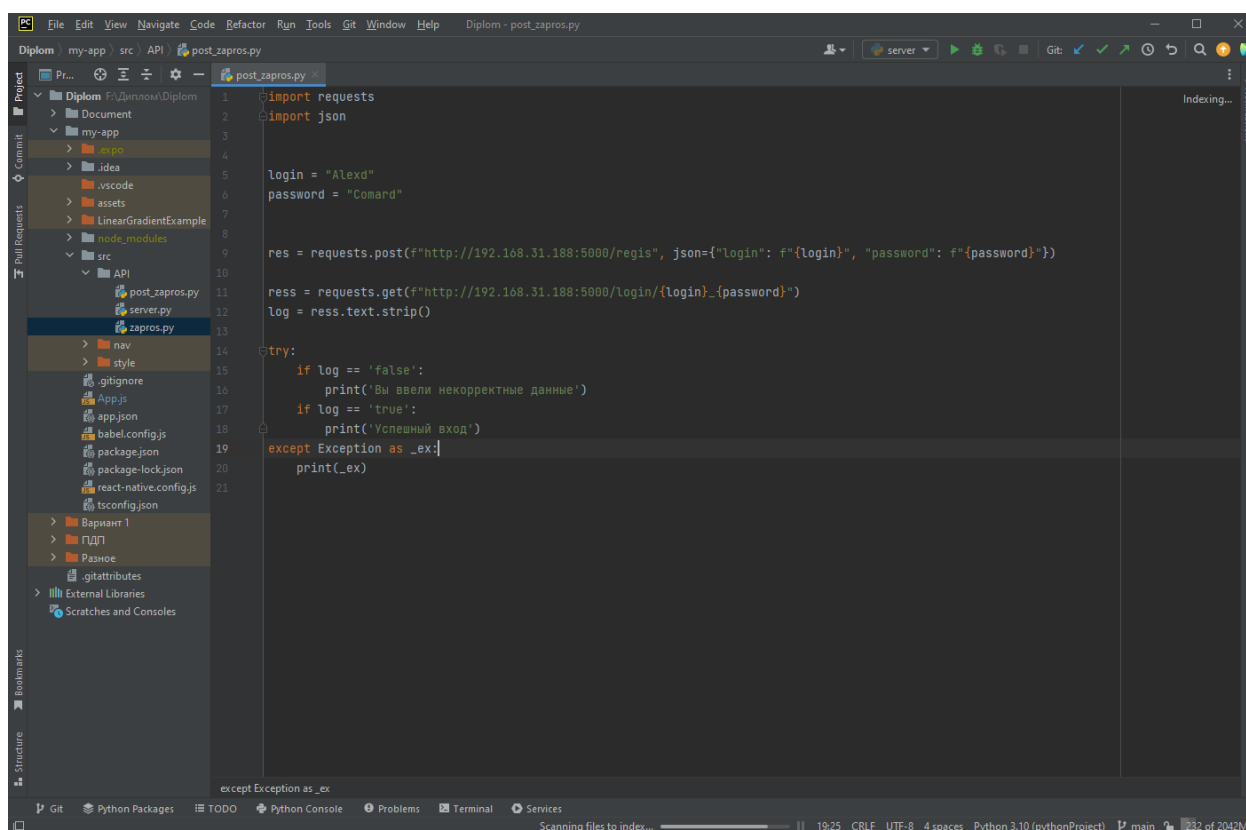


Рисунок 19 –Интерфейс PyCharm с примером кода на Python

Для лучшего понимания каким должен быть интерфейс разрабатываемого программного продукта необходимо было спроектировать макет приложения. В качестве инструмента проектирования был выбран облачный сервис Figma.

					ТПЖА.09.02.07.695577 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		36

Figma — это облачный инструмент для проектирования и совместной работы, используемый дизайнерами, разработчиками и продуктовыми группами для создания совместного использования пользовательских интерфейсов, прототипов и систем проектирования. Он позволяет нескольким пользователям сотрудничать и работать над одними и теми же файлами дизайна в режиме реального времени и имеет расширенные функции для контроля версий, комментирования и передачи дизайна.

Одной из ключевых особенностей Figma является инструмент векторного дизайна, который позволяет пользователям создавать масштабируемые и адаптивные проекты, которые можно легко редактировать и адаптировать для разных устройств и платформ. Figma также имеет надежную библиотеку компонентов и шаблонов дизайна, что позволяет легко создавать согласованные и целостные проекты.

В целом Figma — это мощный и универсальный инструмент проектирования, который может помочь упростить весь процесс проектирования, от идеи и прототипирования до совместной работы и передачи.

Интерфейс при работе с проектом в Figma представлен на рисунке 20.

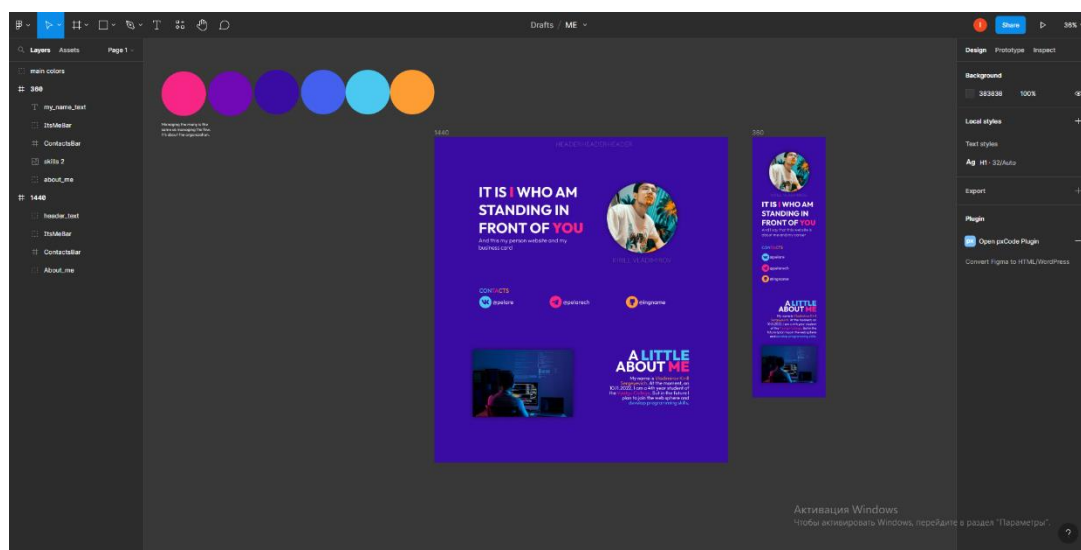


Рисунок 20 – Интерфейс Figma

В качестве СУБД был выбран pgAdmin — популярный инструмент администрирования и управления базами данных PostgreSQL с открытым исходным кодом. Он позволяет пользователям управлять, отслеживать и манипулировать базами данных и серверами PostgreSQL с помощью простого в использовании графического пользовательского интерфейса, который представлен на рисунке 21.

Некоторые из ключевых функций pgAdmin включают в себя:

1) Кроссплатформенная поддержка: pgAdmin можно использовать в операционных системах Windows, macOS и Linux.

2) Поддержка нескольких версий PostgreSQL: pgAdmin поддерживает несколько версий PostgreSQL, от 9.2 до последней версии.

3) Управление базой данных: пользователи могут создавать/удалять базы данных, таблицы, представления, индексы и триггеры, а также управлять схемами и ролями.

4) Редактор SQL: pgAdmin позволяет пользователям писать и выполнять SQL-запросы и сценарии в инструменте.

5) Резервное копирование и восстановление: пользователи могут создавать резервные копии и восстанавливать свои базы данных PostgreSQL либо вручную, либо с помощью запланированного задания.

6) Зависимости объектов: pgAdmin показывает зависимости между объектами базы данных, что упрощает управление сложными структурами базы данных.

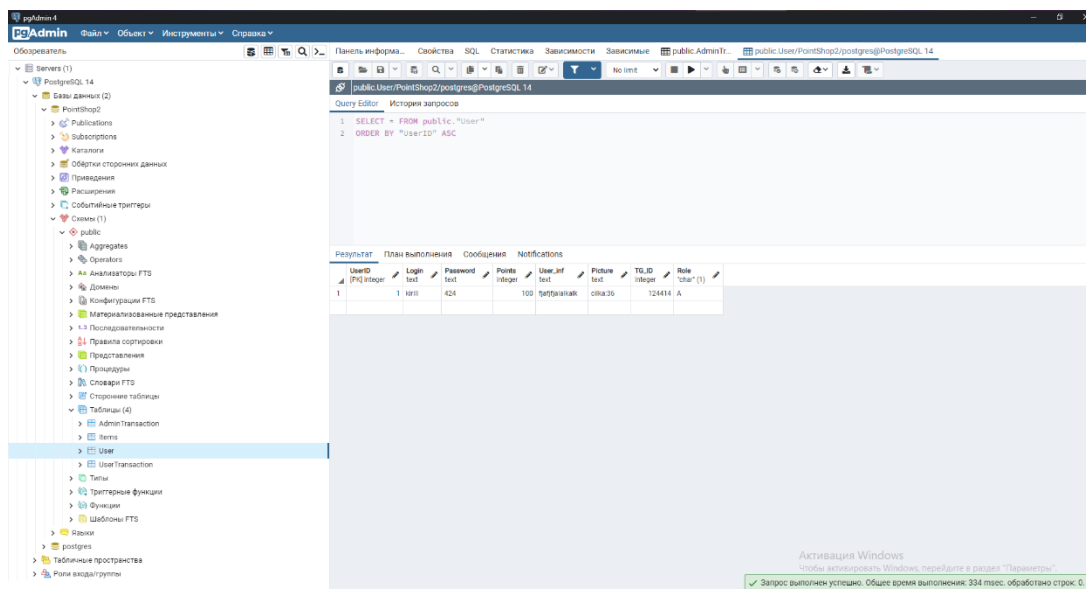


Рисунок 21 – Интерфейс pgAdmin

В целом pgAdmin — это мощный инструмент для администраторов и разработчиков баз данных PostgreSQL, предоставляющий удобный интерфейс для управления базами данных и серверами.

Так как создание приложения с архитектурой клиент-сервер без различных инструментов что уже созданы для этого, это очень трудоемкий и долгий процесс было решено использовать следующие фреймворки и библиотеки:

Для создания мобильного приложения с использованием React Native была выбрана платформа Expo. Он предоставляет набор инструментов и сервисов, упрощающих процесс разработки и публикации мобильных приложений для iOS и Android.

Expo включает в себя множество готовых компонентов и API-интерфейсов, которые позволяют разработчикам легко добавлять такие функции, как камера, карты и уведомления, без необходимости написания собственного кода. Он также предлагает мощный интерфейс командной строки, который позволяет разработчикам быстро настраивать свои проекты и управлять ими.

Помимо инструментов разработки, Expo предоставляет облачную платформу для создания и распространения приложений. Это позволяет разработчикам легко делиться своими приложениями с другими, даже не имея доступа к Apple App Store или Google Play Store.

Инструмент Expo — отличный выбор для тех, кто хочет быстро и легко создавать высококачественные мобильные приложения с помощью React Native.

Далее в качестве библиотеки для создания сервера на Python был выбран Flask. Flask — это микровеб-фреймворк, написанный на Python. Он разработан, чтобы быть легким и простым в использовании, что делает его популярным среди разработчиков, которые хотят быстро создавать веб-приложения.

Flask построен на основе набора инструментов Werkzeug и механизма шаблонов Jinja2. Werkzeug предоставляет низкоуровневую функциональность для обработки HTTP-запросов и ответов, а Jinja2 предоставляет мощный механизм шаблонов, который позволяет разработчикам легко создавать динамические HTML-страницы. Flask также включает в себя встроенный сервер разработки, упрощающий тестирование и отладку приложений в процессе разработки.

В целом Flask — это мощная и гибкая микровеб-инфраструктура, которая хорошо подходит для создания веб-приложений малого и среднего размера.

Для подключения и взаимодействия с базой данных с сервера, решено было использовать библиотеку psycopg2. Psycopg2 — это популярная библиотека Python, которая предоставляет адаптер базы данных PostgreSQL. Библиотека psycopg2 имеет два основных объекта: объект соединения и объект курсора. Объект подключения используется для подключения к базе данных PostgreSQL и управления подключением.

Объект курсора используется для выполнения команд SQL и извлечения данных из базы данных.

В заключение `psycopg2` — это мощная и гибкая библиотека Python для взаимодействия с базами данных PostgreSQL. Он предоставляет простой в использовании интерфейс для подключения к базе данных, выполнения команд SQL и выборки данных. Его обширная документация и поддержка сообщества делают его популярным выбором для разработчиков, работающих с базами данных PostgreSQL в Python.

3.2 Разработка клиентской части

Для разработки клиента было необходимо сделать приложение со страницами авторизации, главной страницей с играми, страница с самой игрой и страницу с отображением информации о пользователе.

Первым делом было необходимо создать проект с помощью `expo` и отобразить результат на мобильном устройстве. Запуск проекта производится командой `npm run start` в командную строку, находясь в основной папке приложения `my-app`, для пояснения `npm` [8] — это менеджер пакетов, после чего на мобильном устройстве необходимо было заранее установить приложение от `expo`, в котором можно было отсканировать `qr-code`, который выводится в командную строку.

После проверки совместимости и отображения результатов проекта на мобильном устройстве, было проведено редактирование файла `App.js`, который генерируется при создании проекта с помощью `expo`. Содержимое файла `App.js` после редактирования, представлено на рисунке 22.

```

my-app > JS App.js > App
1  import React from 'react'
2  import { useFonts } from 'expo-font';
3  import {NavigationContainer} from '@react-navigation/native';
4  import {createNativeStackNavigator} from '@react-navigation/native-stack';
5  import ProfileScreen from './src/nav/Autorization/auth';
6  import HomeScreen from './src/nav/hello_window';
7  import MainScreen from './src/nav/Main/Main_window'
8  import Registration from './src/nav/Autorization/reg';
9  import SudokuScreen from './src/nav/Game/Sudoku_window';
10
11
12  const Stack = createNativeStackNavigator();
13
14  export default function App () {
15
16    const [fontsLoaded] = useFonts({
17      'OpenSansBold': require('./assets/fonts/OpenSansBold.ttf'),
18    });
19    if (!fontsLoaded) {
20      return null;
21    }
22    return (
23      <NavigationContainer>
24        <Stack.Navigator>
25          <Stack.Screen
26            name="Home"
27            component={HomeScreen}
28            options={{headerShown: false}}
29          />
30          <Stack.Screen
31            name="MainScreen"
32            component={MainScreen}
33            options={{headerShown: false}}
34          />
35          <Stack.Screen
36            name="Profile"
37            component={ProfileScreen}
38            options={{headerShown: false}}
39          />
40          <Stack.Screen
41            name="Registration"
42            component={Registration}
43            options={{headerShown: false}}
44          />
45          <Stack.Screen
46            name="Sudoku"
47            component={SudokuScreen}
48            options={{headerShown: false}}
49          />
50        </Stack.Navigator>
51      </NavigationContainer>
52    )
53  }
54

```

Рисунок 22 – Содержимое файла App.js

Основное содержимое файла App.js, является импорт библиотек, импорт файлов с другими окнами и настройка навигации в приложении, а также подключение основного шрифта.

Далее были созданы два файла auth.js и reg.js, в которых прописана логика работы с сервером. В auth.js прописано отображение страницы авторизации, аналогично и в reg.js. Из интересного можно выделить функцию отправки запроса серверу на проверку соответствия в базе данных логина и пароля с данными вводимые пользователем, данная функция представлена на рисунке 23.

```
const getMoviesFromApi = async () => {
  if (text == '' || text2 == ''){
    errorTextEdit2()
  }
  else{
    const res = await fetch ('http://192.168.31.188:5000/login/'+text+'_'+text2).then(response => response.text()).catch((error) => {
      console.error(error);
    });
    if (res.replace("\n", "") == 'true'){
      navigation.navigate('MainScreen')
    }
    else{
      errorTextEdit()
    }
  }
};
```

Рисунок 23 – Реализация функции get запроса на сервер

В представленной функции сначала идет проверка на то имеются ли данные в полях, которые отображаются в клиенте, если данные имеются в обоих полях, то идет отправка запроса серверу, и от него же следует ответ, где клиент снова делает проверку, если ответ полученный от сервера true, то авторизация считается успешно пройденной и используя функцию navigate перемещает клиент на окно с главной страницей, иначе же выводит текст с ошибкой для клиента.

В файле reg.js имеется схожая структура, но уже с проверкой имеется ли пользователь в БД с введенными пользователем данными, если да, то выведет сообщение с ошибкой, иначе же сделает запрос серверу с отправкой json файла, в котором записан логин и пароль введенные пользователем и переведет его на главную страницу. Реализация кода приведена на рисунке 24.

```

const getMoviesFromApi = async () => {
  if (text == '' || text2 == ''){
    errorTextEdit2()
  }
  else{
    const res = await fetch ('http://192.168.31.188:5000/regis',{
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({"login": text, "password": text2,})
    }).then(response => response.text()).catch((error) => {
      console.error(error);
    });
    if (res.replace("\n", "") == 'true'){
      navigation.navigate('MainScreen')
    }
    else{
      errorTextEdit()
    }
  }
};

```

Рисунок 24 – Реализация функции post запроса на сервер

Следующим этапом стала разработка игры и ее отображение. В качестве игры был выбран тест из сборника логических задач и головоломок от Суганяки С. Н. [2]. Его суть состояла в пересчете чисел на изображении по порядку за определенное количество времени, сами же числа не по порядку. Тест был изменен под игру и пользователю необходимо было как можно быстрее нажать на цифры по порядку, от 1 до 50, где после 50 считается победа, если же нажать число не по порядку, то вылезит предупреждение и происходит обновление местоположения цифр, как бы наказывая игрока.

Из интересного в коде с игрой можно выделить функцию перетасовки Фишера – Йейтса [9], которая представляет из себя алгоритм перетасовки конечной последовательности. Алгоритм берет список всех элементов последовательности и постоянно определяет следующий элемент в перетасованной последовательности, случайным образом вытягивая элемент из списка до тех пор, пока элементы не останутся. Алгоритм производит беспристрастную перестановку: каждая перестановка равновероятна.

Алгоритм в виде кода представлен на рисунке 25.

```
function shuffle(array) {
  let currentIndex = array.length,  randomIndex;
  while (currentIndex !== 0) {
    randomIndex = Math.floor(Math.random() * currentIndex);
    currentIndex--;
    [array[currentIndex], array[randomIndex]] = [
      array[randomIndex], array[currentIndex]];
  }
  return array;
}

let numbers = Array.from({length: 50}, (_, i) => i + 1);

shuffle(numbers)
```

Рисунок 25 – Алгоритм генерации случайных перестановок Фишера-Йейтса

Так же в переменную numbers записывается массив чисел от 1, до 50, которую используют в функции с перестановкой. На рисунке 26 представлен код с логикой игры.

```
const SudokuScreen = ({navigation}) => {
  const [text_error, setMyText] = React.useState("");
  let ing = 0
  function checks(number) {
    let num = number
    if (mas_bum[ing]==50){
      setMyText("Вы выиграли")
      setTimeout(() => {
        setMyText("");
        getMoviesback()
      }, 2000);
    }
    if (num == mas_bum[ing]) {
      ing+=1
      setNewColor()
    }
    else{
      ing = 0
      shuffle(numbers)
      setMyText("Число не по порядку")
      setTimeout(() => {
        setMyText("");
      }, 2000);
    }
  }
}
```

Рисунок 26 – Часть кода логики игры

На рисунке 27 показан код react native для отображения экрана приветствия при открытии приложения, после происходит переход на экран авторизации часть кода которого представлена на рисунке 28.

```
return (
  <View style = {stylesheets.style_App_welcome} >
    <LinearGradient
      colors={['rgba(52,180,220,1)', 'rgba(67, 97, 238, 1)']}
      style={stylesheets.background}
    />
    <View style = {{flex:1,justifyContent:"center", alignItems: "center"}}>
      <Text style = {stylesheets.style_FOCUS}>
        FOCUS
      </Text>
    </View>
  </View>
);
};
```

Рисунок 27 – React Native код для отображения экрана приветствия

```
return (
  <View style = {{flex:1}}>
    <View style = {{alignItems: "center", top: 130}}>
      <GradientText style={stylesheets.style_FOCUS_2}>FOCUS</GradientText>
    </View>
    <View style = {{top: 214, left: 39, width: 282.7, height: 536}}>
      <View style = {{width: 315, height: 25.8, alignItems: "center"}}>
        <Text style={{fontSize:16}}
          onChangeText={setMyText}
          value={myText}>
          {myText}
        </Text>
      </View>
      <View style = {{left: 10,top: 20, width: 47.12, height: 19.35, alignItems: "center"}}>
        <Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)}}>
          Логин
        </Text>
      </View>
      <TextInput style={stylesheets.input}
        onChangeText={onChangeText}
        value={text}
        placeholder="Введите логин"
      >
      </TextInput>
      <View style = {{left: 10, top:40, width: 56, height: 19.35, alignItems: "center"}}>
        <Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)}}>
          Пароль
        </Text>
      </View>
    </View>
  </View>
);
```

Рисунок 28 – Часть кода отображения окна авторизации

На рисунке 28 в коде присутствует элемент GradientText, который не является элементом React Native, а был создан для отображения текста с градиентом, класс импортируется из файла GradientText.js, код которого представлен на рисунке 29.

```
import React from 'react';
import {Text} from 'react-native';
import {LinearGradient} from 'expo-linear-gradient';
import MaskedView from '@react-native-masked-view/masked-view';

const GradientText = props => {
  return (
    <MaskedView maskElement={<Text {...props} />}>
      <LinearGradient
        colors={['rgba(52,180,220,1)', 'rgba(67, 97, 238, 1)']}
        start={{x: 0, y: 0}}
        end={{x: 0, y: 1}}>
        <Text {...props} style={[props.style, {opacity: 0}]} />
      </LinearGradient>
    </MaskedView>
  );
};

export default GradientText;
```

Рисунок 29 – Код с добавлением элемента для создания текста с градиентом

На рисунках 30, 31 представлен интерфейс различных страниц программного продукта

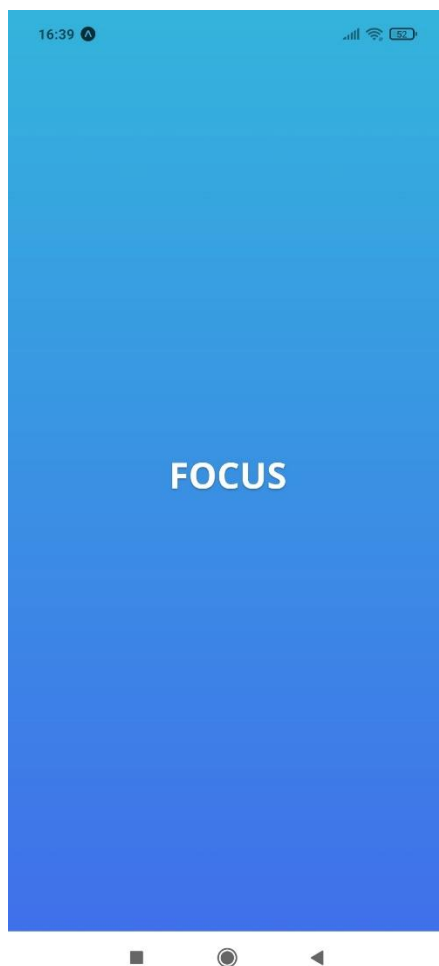


Рисунок 30 – Интерфейс окна приветствия

					ТПЖА.09.02.07.695577 ПЗ	Лист
						48
Изм.	Лист	№ докум.	Подпись	Дата		

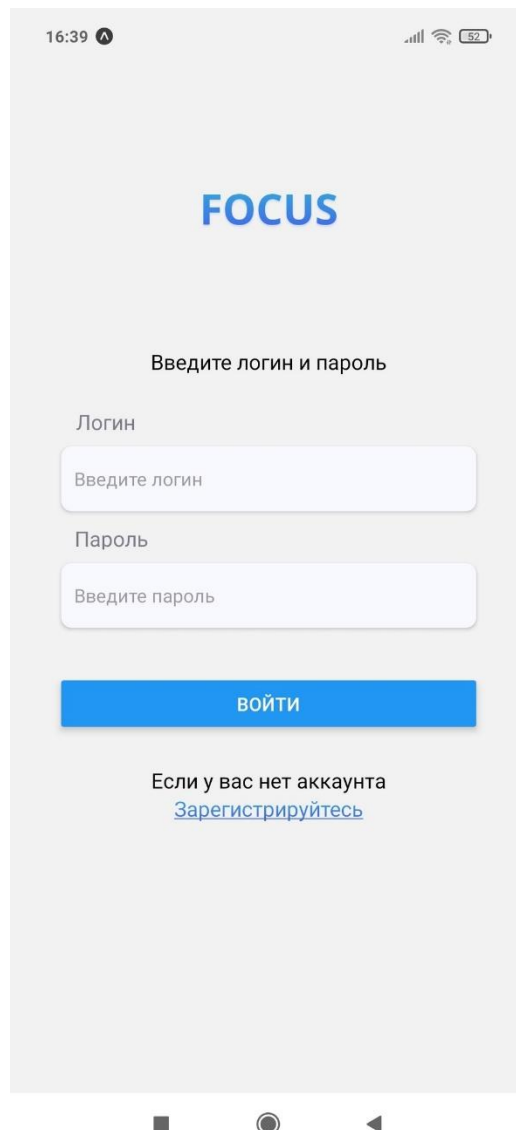


Рисунок 31 – Интерфейс окна с авторизацией

3.3 Разработка серверной части

Разработка серверной части требовала в себе такие вещи как веб приложение для обмена данными через протокол HTTP и подключения к базе данных для ввода и вывода данных.

Сама база данных включает в себя лишь одну таблицу “пользователи”, которая имеет столбцы: id, login, password. Более база данных не имеет таблиц, так как клиент на этапе разработки большего не требует, но при дальнейшем дополнении приложения новым контентом, требуемые таблицы могут быть спокойно созданы и СУБД pgAdmin это позволяет. Таблица Users представлена на рисунке 32.

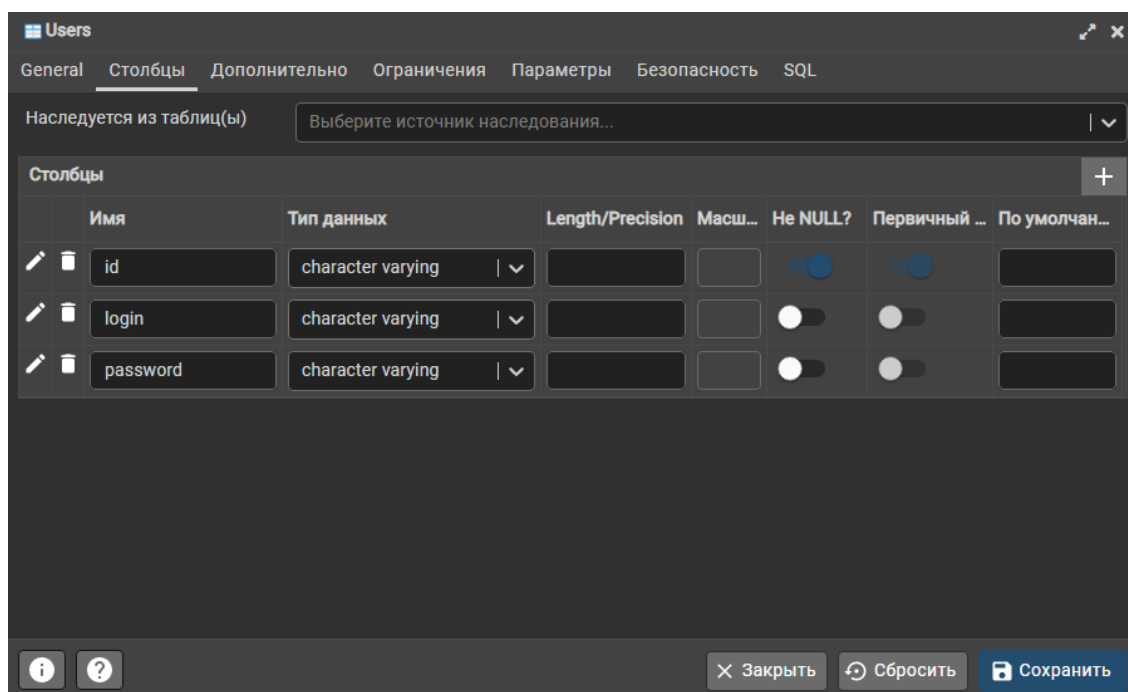


Рисунок 32 – таблица Users в pgAdmin

Первым этапом для создания сервера стало создание класса объекта веб приложения на основе класса Flask из одноименной библиотеки. Вторым же этапом стало подключение к базе данных, через библиотеку psycopg2 и ее функцию “connect ()” в которую нам необходимо было передать данные для подключения к БД, такие как название БД, имя пользователя, пароль и IP. В качестве IP адреса, был использован локальный адрес [10].

Часть кода, описанного выше, можно просмотреть на рисунке 33.

```

from flask import Flask, jsonify, abort, request
import psycopg2

app = Flask(__name__)

connection = psycopg2.connect(
    dbname='diplom',
    user='postgres',
    password='7393',
    host='localhost'
)

if __name__ == '__main__':
    app.run(debug=True, host="192.168.31.188")

```

Рисунок 33 – Основа для создания локального сервера

Следующим этапом стало прописывание маршрутов (URL) и привязка к ним функций. Это делается через декоратор [11] `app.route`, в котором прописывается маршрут и метод, после чего можно создавать функцию и работать с ней. Чтобы работать с базой данных, необходимо сделать курсор, и через метод `execute` выполнять PostgreSQL команды.

Первым делом было решено реализовать функцию, которая будет получать логин и пароль, делать запрос в базу данных с поиском таких логина и пароля и возвращать либо `True` если пользователь с такими данными существует, либо `False` если данные не верны, данная реализация представлена на рисунке 34.

```

ingname
@app.route('/login/<string:login_name>_<string:password>', methods=['GET'])
def get_id(login_name, password):
    cur = connection.cursor()
    cur.execute(
        f''' SELECT login, password FROM public."Users" WHERE login = '{login_name}' and password = '{password}' '''
    )
    users = cur.fetchone()
    if users is None:
        return jsonify(False)
    else:
        log = users[0]
        passw = users[1]
        if log == '' or passw == '':
            return jsonify(False)
        if log != '' and passw != '':
            return jsonify(True)
        else:
            abort(404)
    cur.close()

```

Рисунок 34 – реализация функции проверки данных

Данная функция является проверкой логина и пароля введенные пользователем и теми данными, что занесены в базу данных при условии, если данные совпадают, то возвращает ответ в виде веб страницы со словом true, либо при несовпадении данных false.

Строка “users = cur.fetchone()”, извлекает первую строку результирующего набора, возвращаемого запросом и записывает результат в переменную users.

Следующей задачей стала реализация функции, в которой идет считывание json файла, переданного по маршруту и создание записи в базу данных или если более обобщенно, то написание функции регистрации пользователя в БД.

Для этого необходимо было прописать новый декоратор, новый маршрут к нему и выбрать для него метод POST, после чего записать новую функцию. Данная реализация приведена на рисунке 35.

```

ingname
@app.route('/regis', methods=['POST'])
def add_new_user():
    content = request.json
    cur = connection.cursor()
    cur.execute(f''' SELECT login FROM public."Users" WHERE login = '{content['login']}' ''')
    res = cur.fetchone()
    if res is None:
        cur.execute(
            f''' SELECT id FROM public."Users" '''
        )
        last_id = int(cur.fetchall()[-1][0]) + 1
        cur.execute(
            f''' INSERT INTO public."Users"( id, login, password, role) VALUES ('{str(last_id)}',
                                                    '{content['login']}', '{content['password']}', '2'); '''
        )
        connection.commit()
        cur.close()
        return jsonify(True), 201
    else:
        return jsonify(False)

```

Рисунок 35 – Реализация функции записи новых данных

Строка “content = request.json” выполняет запрос на получение json файла который необходимо передавать через запрос, ведь именно в нем должны быть логин и пароль на добавление пользователя в БД.

Далее производится запрос в базу данных на соответствие с логином, который был передан через json файл и делается проверка на идентичность данных. Это необходимо для того, чтобы понять, существует ли уже в базе данных пользователь с таким же логином, если такого нет, то он идет дальше, иначе отправляет в ответ False, что будет означать, что пользователь с таким логином уже есть в системе.

После проверки на логин делается запрос в бд на получение id у последнего пользователя, затем создается новая переменная и присваивается последнее значение id увеличенное на 1, после чего делается новый запрос в базу данных на добавление новой записи в таблицу Users, в которой добавляется новый id, логин и пароль от пользователя затем делается commit по одноименной функции. В конце отправляется ответ в виде строки со строкой true.

Таким образом были реализованы основные методы, которые необходимы для клиента, на стороне сервера.

					ТПЖА.09.02.07.695577 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		53

3.4 Тестирование программного продукта

В качестве мобильного устройства для проверки работоспособности клиентской части был использован смартфон Xiaomi POCO X3, поэтому отображение элементов клиента на других устройствах может отличаться.

Тестирование приложения было проведено в виде тест кейсов и их результат отображены на таблицах 2-6.

Таблица 2 – Тест кейс 1

Наименование	Описание
Наименование проекта	Тестирование мобильного приложения FOCUS
Номер версии	1.0
Имя тестера	Владимиров Кирилл
Даты тестирования	01.06.2023
Test Case #	ТС-1
Приоритет тестирования (Малый/Средний/высокий)	Низкий
Название тестирования/Имя	Тестирование авторизации, пустые данные.
Резюме испытания	При тестировании нужно проверить как отреагирует приложение при авторизации с пустыми данными
Шаги тестирования	Шаг 1. Запустить приложение Шаг 2. Нажать на кнопку авторизации
Ожидаемый результат	Вывод сообщения о том, что данные для авторизации не были введены
Фактический результат	Вывод сообщения “Вы не ввели данные”
Статус (Pass/Fail)	pass

Таблица 3 – Тест кейс 2

Наименование	Описание
Наименование проекта	Тестирование мобильного приложения FOCUS
Номер версии	1.0
Имя тестера	Владимиров Кирилл
Даты тестирования	01.06.2023
Test Case #	ТС-2
Приоритет тестирования (Малый/Средний/высокий)	Средний
Название тестирования/Имя	Тестирование авторизации, ввод данных о не существующем пользователе
Резюме испытания	При тестировании нужно проверить как отреагирует приложение при авторизации с данными о не существующем пользователе в системе
Шаги тестирования	Шаг 1. Запустить приложение Шаг 2. Ввести в поле для логина "Text" Шаг 3. Ввести в поле для пароля "Text" Шаг 4. Нажать на кнопку авторизации
Ожидаемый результат	Вывод сообщения о том, что данные для авторизации не верны
Фактический результат	Вывод сообщения "Вы ввели не правильные данные"
Статус (Pass/Fail)	pass

Таблица 4 – Тест кейс 3

Наименование	Описание
Наименование проекта	Тестирование мобильного приложения FOCUS
Номер версии	1.0
Имя тестера	Владимиров Кирилл
Даты тестирования	01.06.2023
Test Case #	ТС-3
Приоритет тестирования (Малый/Средний/высокий)	Средний
Название тестирования/Имя	Тестирование авторизации, ввод данных о существующем пользователе
Резюме испытания	При тестировании нужно проверить как отреагирует приложение при авторизации с правильными для ввода данными
Шаги тестирования	Шаг 1. Запустить приложение Шаг 2. Ввести в поле для логина “Admin” Шаг 3. Ввести в поле для пароля “Admin” Шаг 4. Нажать на кнопку авторизации
Ожидаемый результат	Нас перенаправит на главную страницу
Фактический результат	Нас перенаправляет на главную страницу
Статус (Pass/Fail)	pass

Таблица 5 – Тест кейс 4

Наименование	Описание
Наименование проекта	Тестирование мобильного приложения FOCUS
Номер версии	1.0
Имя тестера	Владимиров Кирилл
Даты тестирования	01.06.2023
Test Case #	ТС-4
Приоритет тестирования (Малый/Средний/высокий)	Средний
Название тестирования/Имя	Тестирование регистрации, ввод данных о новом пользователе
Резюме испытания	При тестировании нужно проверить как отреагирует приложение при регистрации нового пользователя
Шаги тестирования	Шаг 1. Запустить приложение Шаг 2. Ввести в поле для логина “Artur” Шаг 3. Ввести в поле для пароля “74984” Шаг 4. Нажать на кнопку регистрации
Ожидаемый результат	Нас перенаправит на главную страницу
Фактический результат	Нас перенаправляет на главную страницу
Статус (Pass/Fail)	pass

Таблица 6 – Тест кейс 5

Наименование	Описание
Наименование проекта	Тестирование мобильного приложения FOCUS
Номер версии	1.0
Имя тестера	Владимиров Кирилл
Даты тестирования	01.06.2023
Test Case #	TC-5
Приоритет тестирования (Малый/Средний/высокий)	Средний
Название тестирования/Имя	Тестирование регистрации, ввод данных о существующем пользователе
Резюме испытания	При тестировании нужно проверить как отреагирует приложение при регистрации существующего пользователя
Шаги тестирования	Шаг 1. Запустить приложение Шаг 2. Ввести в поле для логина "Artur" Шаг 3. Ввести в поле для пароля "74984" Шаг 4. Нажать на кнопку регистрации
Ожидаемый результат	Нам сообщат о том, что такой логин занят
Фактический результат	Нам сообщают "Этот логин уже занят"
Статус (Pass/Fail)	pass

ЗАКЛЮЧЕНИЕ

В данной дипломной работе были проведены исследования и анализ предметной области, проектирование и разработка, направленные на создание мобильного приложения для развития концентрации внимания. В процессе работы были достигнуты следующие результаты:

1) Анализ предметной области: В главе, посвященной анализу предметной области, был осуществлен обзор существующих решений и аналогов, разработанного приложения. Было выявлено, что большинство аналогов не локализованы на русский язык и большинство функционала доступно только на платной основе.

2) Проектирование программного приложения: В главе, посвященной проектированию, были определены основные требования к разрабатываемому приложению. Был выбран тип архитектуры клиент-сервер. Спроектирована структура сервера, а также файловая структура. Спроектирован интерфейс клиента и заключительным этапом была проведена разработка алгоритмов функционирования ПО.

3) Программная реализация приложения: В главе, посвященной разработке, был проведен окончательный выбор и основания средств разработки. Проведена разработка клиентской части с отрисовкой интерфейса на мобильном устройстве и ее логики, а также проведена разработка серверной части с обработкой запросов от клиента. И заключительным этапом стало тестирование путем использования тест кейсов.

В конце хотелось бы отметить, что есть возможности и пути реализации для дальнейшего развития этого проекта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) **Батранусов, Г.А.** Архитектура «Клиент-Сервер» / Г.А. Батранусов. - Текст: электронный // Itelon. - URL: <https://itelon.ru/blog/arkhitektura-klient-server/> (дата обращения: 10.06.2023).

2) **Суганяки, С. Н.** Сборник логических задач и головоломок / С. Н. Суганяки. - 2-е изд. -: Москва: Готанд, 2016. - 104 с. - Текст: непосредственный.

3) **Колосов П. И.** Протокол HTTP / П.И. Колосов. - Текст: электронный // QaBible. - URL: https://vladislavermeev.gitbook.io/qa_bible/seti-i-okolomikh/http (дата обращения: 15.06.2023).

4) **Петухов А.С.** React Native Project Structure: A Best Practices Guide. / А.С. Петухов. - Текст: электронный // Waldo. - URL: <https://www.waldo.com/blog/react-native-project-structure> (дата обращения: 10.06.2023).

5) **Трасковский А.** Обзор языка разметки HTML для новичков / А. Трасковский. - Текст: электронный // Timeweb. - URL: <https://timeweb.com/ru/community/articles/chto-takoe-html> (дата обращения: 10.06.2023).

6) **Коньшев В. А.** Микрофреймворки / В.А. Коньшев. - Текст: электронный // Hexlet. - URL: https://ru.hexlet.io/courses/python-overview-of-web-development/lessons/microframework/theory_unit (дата обращения: 10.06.2023).

7) **Евкова А.** Алгоритмизация как обязательный этап разработки программы (Разработка программ) / А. Евкова. - Текст: электронный // Evkova. - URL: <https://www.evkova.org/kursovye-raboty/algoritmizatsiya-kak-obyazatelnyij-etap-razrabotki-programmyi-razrabotka-programm> (дата обращения: 10.06.2023).

8) **Германов Н. А.** Что такое npm / Н.А. Германов. - Текст: электронный // Справочник Node.js. - URL: <https://nodejsdev.ru/guides/webdraftt/npm/> (дата обращения: 10.06.2023).

9) **Лергентов П.А.** Метод генерации случайной перестановки, алгоритм Фишера-Йетса / П.А. Лергентов. - Текст: электронный // Neerc. - URL: https://neerc.ifmo.ru/wiki/index.php?title=Метод_генерации_случайной_перестановки,_алгоритм_Фишера-Йетса (дата обращения: 10.06.2023).

10) **Василенко Д. С.** Что такое локальный (внутренний) IP адрес / Д. С. Василенко. - Текст: электронный // Vpnhook. - URL: <https://vpnhook.com/chto-takoe-localniy-vnutrenniy-ip-adres/> (дата обращения: 10.06.2023).

11) **Гардымов С. А.** Декораторы в Python: понять и полюбить / С. А. Гардымов. - Текст: электронный // Tproger. - URL: <https://tproger.ru/translations/demystifying-decorators-in-python/> (дата обращения: 10.06.2023).

ПРИЛОЖЕНИЕ А

ОТЛАДКА КОДА

```
App.js
import React from 'react'
import { useFonts } from 'expo-font';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import ProfileScreen from './src/nav/Autorization/auth';
import HomeScreen from './src/nav/hello_window';
import MainScreen from './src/nav/Main/Main_window';
import Registration from './src/nav/Autorization/reg';
import SudokuScreen from './src/nav/Game/Sudoku_window';
import UserScreen from './src/nav/Main/User_window';

const Stack = createNativeStackNavigator();

export default function App () {

  const [fontsLoaded] = useFonts({
    'OpenSansBold': require('./assets/fonts/OpenSansBold.ttf'),
  });

  if (!fontsLoaded) {
    return null;
  }

  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ headerShown: false }}
        />
        <Stack.Screen
          name="MainScreen"
```

Продолжение Приложения А

```
        component={MainScreen}
        options={{headerShown: false}}
      />
      <Stack.Screen
        name="Profile"
        component={ProfileScreen}
        options={{headerShown: false}}
      />
      <Stack.Screen
        name="Registration"
        component={Registration}
        options={{headerShown: false}}
      />
      <Stack.Screen
        name="Sudoku"
        component={SudokuScreen}
        options={{headerShown: false}}
      />
      <Stack.Screen
        name="User"
        component={UserScreen}
        options={{headerShown: false}}
      />
    </Stack.Navigator>
  </NavigationContainer>
)
}
```

Hellow_window.js

```
import stylesheets from '../style/style'
import { LinearGradient } from 'expo-linear-gradient';
import { Text, View } from 'react-native'
```

Продолжение Приложения А

```
const HomeScreen = ({navigation}) => {
  setTimeout(() => {
    navigation.navigate('Profile');
  }, 2500);
  return (
    <View style = {stylesheets.style_App_welcome} >
      <LinearGradient
        colors={['rgba(52,180,220,1)', 'rgba(67, 97, 238, 1)']}
        style={stylesheets.background}
      />
      <View style = {{flex:1,justifyContent:"center", alignItems: "center"}}>
        <Text style = {stylesheets.style_FOCUS}>
          FOCUS
        </Text>
      </View>
    </View>
  );
};
```

```
export default HomeScreen;
```

auth.js

```
import {useBackHandler} from '@react-native-community/hooks';
import {View, Text, Button, TextInput} from 'react-native'
import GradientText from './GradientText';
import stylesheets from '../style/style'
import React from 'react';
```

```
let login
```

```
const ProfileScreen = ({navigation}) => {
  useBackHandler(() => {
    if (true) {
```

Продолжение Приложения А

```
    return true
  }
})
const [text, onChangeText] = React.useState("");
const [text2, onChangeText2] = React.useState("");
const [myText, setMyText] = React.useState("Введите логин и пароль");
function a (){
  login = text
}

const getMoviesFromApi = async () => {
  if (text == "" || text2 == ""){
    errorTextEdit2()
  }
  else{
    const res = await fetch(
      ('http://192.168.31.188:5000/login/'+text+'_'+text2).then(response
      response.text()).catch((error) => {
        console.error(error);

      });
    if (res.replace("\n", "") == 'true'){
      a()
      navigation.navigate('MainScreen')
    }
    else{
      errorTextEdit()
    }
  }
};

const getMoviesInReg = async () => {
  navigation.navigate('Registration')
```

Продолжение Приложения А

```
};  
const errorTextEdit = async () => {  
  setMyText("Вы ввели не правильные данные")  
}  
const errorTextEdit2 = async () => {  
  setMyText("Вы не ввели данные")  
}  
return (  
  <View style = {{flex:1}}>  
    <View style = {{alignItems: "center", top: 130}}>  
      <GradientText style={stylesheets.style_FOCUS_2}>FOCUS</GradientText>  
    </View>  
    <View style = {{top: 214, left: 39, width: 282.7, height: 536}}>  
      <View style = {{width: 315, height: 25.8, alignItems: "center"}}>  
        <Text style={{fontSize:16}}  
          onChangeText={setMyText}  
          value={myText}>  
          {myText}  
        </Text>  
      </View>  
      <View style = {{left: 10,top: 20, width: 47.12, height: 19.35, alignItems:  
"center"}}>  
        <Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)}}>  
          Логин  
        </Text>  
      </View>  
      <TextInput style={stylesheets.input}  
        onChangeText={onChangeText}  
        value={text}  
        placeholder="Введите логин"  
      >  
    </TextInput>  
    <View style = {{left: 10, top:40, width: 56, height: 19.35, alignItems: "center"}}>
```

Продолжение Приложения А

```
<Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)"}}>
    Пароль
</Text>
</View>
<TextInput style={stylesheets.input2}
    onChangeText={onChangeText2}
    value={text2}
    placeholder="Введите пароль"
    >
</TextInput>
<View style = {stylesheets.button_a}>
    <Button style = {{}}
        onPress={()=>
            getMoviesFromApi()
        }
        title="Войти"
    >
</Button>
</View>
<View style = {{width: 315, height: 30, alignItems: "center", top: 120}}>
    <Text style={{fontSize:16}} >
        Если у вас нет аккаунта

    </Text>
    <GradientText    style={{fontSize:16,    textDecorationLine:    'underline'}}
onPress={()=>
        getMoviesInReg()
    }>Зарегистрируйтесь</GradientText>
</View>
</View>
</View>
);
};
```

Продолжение Приложения А

```
export default ProfileScreen;  
export {login}
```

reg.js

```
import {View, Text, Button, TextInput} from 'react-native'  
import GradientText from './GradientText';  
import stylesheets from '../style/style'  
import React from 'react';  
  
const Registration = ({navigation}) => {  
  const [text, onChangeText] = React.useState("");  
  const [text2, onChangeText2] = React.useState("");  
  const [myText, setMyText] = React.useState("Введите логин и пароль");  
  const getMoviesFromApi = async () => {  
    if (text == "" || text2 == ""){  
      errorTextEdit2()  
    }  
    else{  
      const res = await fetch ('http://192.168.31.188:5000/regis',{  
        method: 'POST',  
        headers: {  
          'Content-Type': 'application/json',  
        },  
        body: JSON.stringify({"login": text, "password": text2,})  
      }).then(response => response.text()).catch((error) => {  
        console.error(error);  
      });  
      if (res.replace("\n", "") == 'true'){  
        navigation.navigate('MainScreen')  
      }  
      else{  
        errorTextEdit()  
      }  
    }  
  }  
}
```

Продолжение Приложения А

```
    }  
  }  
};  
const getMoviesInAuth = async () => {  
  navigation.navigate('Profile')  
  
};  
const errorTextEdit = async () => {  
  setMyText("Этот логин уже занят")  
}  
const errorTextEdit2 = async () => {  
  setMyText("Вы не ввели данные")  
}  
return (  
  <View style = {{flex:1}}>  
    <View style = {{alignItems: "center", top: 130}}>  
      <GradientText style={stylesheets.style_FOCUS_2}>FOCUS</GradientText>  
    </View>  
    <View style = {{top: 214, left: 39, width: 282.7, height: 536}}>  
      <View style = {{width: 315, height: 25.8, alignItems: "center"}}>  
        <Text style={{fontSize:16}}onChangeText={setMyText}  
          value={myText}  
        >  
          {myText}  
        </Text>  
      </View>  
      <View style = {{left: 10,top: 20, width: 47.12, height: 19.35, alignItems:  
"center"}}>  
        <Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)}}>  
          Логин  
        </Text>  
      </View>  
      <TextInput style={stylesheets.input}  
        onChangeText={onChangeText}
```

Продолжение Приложения А

```
        value={text}
        placeholder="Введите логин"
      >
    </TextInput>
    <View style = {{left: 10, top:40, width: 56, height: 19.35, alignItems: "center"}}>
      <Text style={{fontSize:16, color: "rgba(124, 124, 138, 1)}}>
        Пароль
      </Text>
    </View>
    <TextInput style={stylesheets.input2}
      onChangeText={onChangeText2}
      value={text2}
      placeholder="Введите пароль"
    >
  </TextInput>

  <View style = {stylesheets.button_a}>
    <Button style = {{}}
      onPress={()=>
        getMoviesFromApi()
      }
      title="Зарегистрироваться"
    >
  </Button>
</View>

  <View style = {{width: 300, height: 30, alignItems: "center", top: 120}}>
    <Text style={{fontSize:16}} >
      Если у вас есть аккаунт

    </Text>
    <GradientText style={{fontSize:16, textDecorationLine: 'underline'}}
      onPress={()=>
```

Продолжение Приложения А

```
      getMoviesInAuth()
    }>Авторизируйтесь</GradientText>
  </View>
</View>
</View>
);
};
```

```
export default Registration;
```

GradientText.js

```
import React from 'react';
import {Text} from 'react-native';
import {LinearGradient} from 'expo-linear-gradient';
import MaskedView from '@react-native-masked-view/masked-view';

const GradientText = props => {
  return (
    <MaskedView maskElement={<Text {...props} />}>
      <LinearGradient
        colors={['rgba(52,180,220,1)', 'rgba(67, 97, 238, 1)']}
        start={{x: 0, y: 0}}
        end={{x: 0, y: 1}}>
        <Text {...props} style={[props.style, {opacity: 0}]} />
      </LinearGradient>
    </MaskedView>
  );
};
```

```
export default GradientText;
```

Game_window

```
import {View, Text, Image, TouchableOpacity} from 'react-native'
import stylesheets from '../style/style'
```

Продолжение Приложения А

```
import React from 'react';

function shuffle(array) {
  let currentIndex = array.length, randomIndex;
  while (currentIndex !== 0) {
    randomIndex = Math.floor(Math.random() * currentIndex);
    currentIndex--;
    [array[currentIndex], array[randomIndex]] = [
      array[randomIndex], array[currentIndex]];
  }
  return array;
}

let mas_bum = Array.from({length: 50}, (_, i) => i + 1);
let numbers = Array.from({length: 50}, (_, i) => i + 1);
shuffle(numbers)

const SudokuScreen = ({navigation}) => {
  const [text_error, setMyText] = React.useState("");
  let ing = 0
  function checks(number) {
    let num = number
    if (mas_bum[ing]===50){
      setMyText("Вы выиграли")
      setTimeout(() => {
        setMyText("");
        getMoviesback()
      }, 2000);
    }
    if (num == mas_bum[ing]) {
      ing+=1
    }
  }
  else{
    ing = 0
    ing = 0
  }
}
```

Продолжение Приложения А

```
ing = 0
shuffle(numbers)
setMyText("Число не по порядку")
setTimeout(() => {
  setMyText("");
}, 2000);
}
}

const getMoviesback = async () => {
  navigation.navigate('MainScreen')
  shuffle(numbers)
};

return (
  <View>
    <View>
      <TouchableOpacity style={{height: 70, width: 70}} onPress={()=>
        getMoviesback()
      }>
        <Image
          source={require('../../assets/img/ico_back.png')}
          style={{width: null, resizeMode: 'contain', height: 50, width: 50, top: 40, left:
20}}
        />
      </TouchableOpacity>
    </View>
    <View style = {{flexDirection: "row", justifyContent: 'center', top:'5%'}}>
      <View style = {{justifyContent: 'flex-start'}}>
        <TouchableOpacity style = {stylesheets.view_cubic} onPress={()=>
checks(numbers[0])}>
          <Text>
            {numbers[0]}
          </Text>

```

Продолжение Приложения А

```
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic} onPress={()=>
checks(numbers[1])}>
  <Text>
    {numbers[1]}
  </Text>
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic}onPress={()=>
checks(numbers[2])}>
  <Text>
    {numbers[2]}
  </Text>
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic}onPress={()=>
checks(numbers[3])}>
  <Text>
    {numbers[3]}
  </Text>
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic}onPress={()=>
checks(numbers[4])}>
  <Text>
    {numbers[4]}
  </Text>
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic}onPress={()=>
checks(numbers[5])}>
  <Text>
    {numbers[5]}
  </Text>
</TouchableOpacity>
<TouchableOpacity style= {stylesheets.view_cubic}onPress={()=>
checks(numbers[6])}>
  <Text>
```

Продолжение Приложения А

```
        {numbers[6]}
      </Text>
    </TouchableOpacity>
    <TouchableOpacity          style=          {stylesheets.view_cubic}onPress={()=>
checks(numbers[7])}>
      <Text>
        {numbers[7]}
      </Text>
    </TouchableOpacity>
    <TouchableOpacity          style=          {stylesheets.view_cubic}onPress={()=>
checks(numbers[8])}>
      <Text>
        {numbers[8]}
      </Text>
    </TouchableOpacity>
    <TouchableOpacity          style=          {stylesheets.view_cubic}onPress={()=>
checks(numbers[9])}>
      <Text>
        {numbers[9]}
      </Text>
    </TouchableOpacity>
  </View>
  <View style = {{justifyContent: 'flex-start'}}>
    <TouchableOpacity          style=          {stylesheets.view_cubic}onPress={()=>
checks(numbers[10])}>
      <Text>
        {numbers[10]}
      </Text>
    </TouchableOpacity>
    <TouchableOpacity          style=          {stylesheets.view_cubic}onPress={()=>
checks(numbers[11])}>
      <Text>
        {numbers[11]}
      </Text>
```

Продолжение Приложения А

```
</TouchableOpacity>
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[12])}>
  <Text>
    {numbers[12]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}    onPress={()=>
checks(numbers[13])}>
  <Text>
    {numbers[13]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}    onPress={()=>
checks(numbers[14])}>
  <Text>
    {numbers[14]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}    onPress={()=>
checks(numbers[15])}>
  <Text>
    {numbers[15]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}    onPress={()=>
checks(numbers[16])}>
  <Text>
    {numbers[16]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}    onPress={()=>
checks(numbers[17])}>
  <Text>
```

Продолжение Приложения А

```
        {numbers[17]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[18])}>
      <Text>
        {numbers[18]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[19])}>
      <Text>
        {numbers[19]}
      </Text>
    </TouchableOpacity >
  </View>
  <View style = {{justifyContent: 'flex-start'}}>
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[20])}>
      <Text>
        {numbers[20]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[21])}>
      <Text>
        {numbers[21]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[22])}>
      <Text>
        {numbers[22]}
      </Text>
```

Продолжение Приложения А

```
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[23])}>
  <Text>
    {numbers[23]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[24])}>
  <Text>
    {numbers[24]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[25])}>
  <Text>
    {numbers[25]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[26])}>
  <Text>
    {numbers[26]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[27])}>
  <Text>
    {numbers[27]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[28])}>
  <Text>
```

Продолжение Приложения А

```
        {numbers[28]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[29])}>
      <Text>
        {numbers[29]}
      </Text>
    </TouchableOpacity >
  </View>
  <View style = {{justifyContent: 'flex-start'}}>
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[30])}>
      <Text>
        {numbers[30]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[31])}>
      <Text>
        {numbers[31]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[32])}>
      <Text>
        {numbers[32]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[33])}>
      <Text>
        {numbers[33]}
      </Text>
```

Продолжение Приложения А

```
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[34])}>
  <Text>
    {numbers[34]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[35])}>
  <Text>
    {numbers[35]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[36])}>
  <Text>
    {numbers[36]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[37])}>
  <Text>
    {numbers[37]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[38])}>
  <Text>
    {numbers[38]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[39])}>
  <Text>
```

Продолжение Приложения А

```
        {numbers[39]}
      </Text>
    </TouchableOpacity >
  </View>
  <View style = {{justifyContent: 'flex-start'}}>
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[40])}>
      <Text>
        {numbers[40]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[41])}>
      <Text>
        {numbers[41]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[42])}>
      <Text>
        {numbers[42]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[43])}>
      <Text>
        {numbers[43]}
      </Text>
    </TouchableOpacity >
    <TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[44])}>
      <Text>
        {numbers[44]}
      </Text>
```

Продолжение Приложения А

```
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[45])}>
  <Text>
    {numbers[45]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[46])}>
  <Text>
    {numbers[46]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[47])}>
  <Text>
    {numbers[47]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[48])}>
  <Text>
    {numbers[48]}
  </Text>
</TouchableOpacity >
<TouchableOpacity          style=      {stylesheets.view_cubic}onPress={()=>
checks(numbers[49])}>
  <Text>
    {numbers[49]}
  </Text>
</TouchableOpacity >
</View>
</View>
<View style = {{bottom: -80}}>
```

Продолжение Приложения А

```
<Text style = {{fontSize: 20, height: '100%', textAlign: 'center'}}
  onChangeText={setMyText}
  value={text_error}>
  {text_error}
</Text>
</View>
</View>
```

```
);
```

```
};
```

```
export default SudokuScreen;
```

Main_window.js

```
import {View, Text, Image, TouchableOpacity} from 'react-native'
import GradientText from '../Autorization/GradientText';
import stylesheets from '../style/style'
import React from 'react';
```

```
const MainScreen = ({navigation}) => {
  const getMoviesGameSudoku = async () => {
    navigation.navigate('Sudoku')
  };
  const getMoviesUserWindow = async () => {
    navigation.navigate('User')
  };
  return (
    <View style = {stylesheets.container}>
      <View style = {{alignItems: "center", top: '4%'}}>
        <GradientText style={stylesheets.style_FOCUS_2}>FOCUS</GradientText>
      </View>
      <View style = {{top: '7%', width: 100,height: 30,alignItems: "center",}}>
```

Продолжение Приложения А

```
<Text style = {stylesheets.text_played}>
  Игры
</Text>
</View>
<View style= {stylesheets.view_container}>
  <View style= {{flexDirection: "row", justifyContent: 'space-evenly'}}>
    <View style = {stylesheets.view_container_game}>
      <TouchableOpacity style={{flex:1, alignItems: "center",}} onPress={()=>
        getMoviesGameSudoku()
      }>
      <Text style={{fontSize:17, color: "rgba(124, 124, 138, 1)", }}>
        Найти числа по порядку
      </Text>
    </TouchableOpacity>
  </View>
</View>
<View style= {{flexDirection: 'row', justifyContent: 'space-around', top: '17%'}}>
  <TouchableOpacity>
    <Image
      source={require('../../assets/img/home_be.png')}
      style={stylesheets.image}
    />
  </TouchableOpacity>
  <TouchableOpacity onPress={()=>
    getMoviesUserWindow()
  }>
    <Image
      source={require('../../assets/img/user_be.png')}
      style={{width: null, resizeMode: 'contain', height: 50, width: 50, top: '4%'}}
    />
  </TouchableOpacity>
</View>
</View>
```

Продолжение Приложения А

```
);  
};
```

```
export default MainScreen;
```

User_window.js

```
import {View, Text, Image, TouchableOpacity} from 'react-native'  
import GradientText from '../Authorization/GradientText';  
import stylesheets from '../style/style'  
import React from 'react';  
import {login} from '../Authorization/auth';  
  
const UserScreen = ({navigation}) => {  
  const getMoviesMainWindow = async () => {  
    navigation.navigate('MainScreen')  
  };  
  return (  
    <View style = {stylesheets.container}>  
      <View style = {{alignItems: "center", top: '4%'}}>  
        <GradientText style={stylesheets.style_FOCUS_2}>{login}</GradientText>  
      </View>  
      <View style = {stylesheets.view_played}>  
        <Text style = {stylesheets.text_played}>  
          Ваша статистика  
        </Text>  
      </View>  
      <View style= {stylesheets.view_container}>  
        <View style= {{flexDirection: "row", justifyContent: 'space-evenly'}}>  
        </View>  
      </View>  
      <View style= {{flexDirection: 'row', justifyContent: 'space-around', top: '17%'}}>  
        <TouchableOpacity onPress={()=>
```

Продолжение Приложения А

```
      getMoviesMainWindow()
    }>
    <Image
      source={require('../../assets/img/home_be.png')}
      style={stylesheets.image}
    />
  </TouchableOpacity>
  <TouchableOpacity>
    <Image
      source={require('../../assets/img/user_be.png')}
      style={{width: null, resizeMode: 'contain', height: 50, width: 50, top: '4%'}}
    />
  </TouchableOpacity>
</View>
</View>
);
};
```

export default UserScreen;

server.py

```
import {View, Text, Image, TouchableOpacity} from 'react-native'
import GradientText from '../Authorization/GradientText';
import stylesheets from '../../style/style'
import React from 'react';
import {login} from '../Authorization/auth';

const UserScreen = ({navigation}) => {
  const getMoviesMainWindow = async () => {
    navigation.navigate('MainScreen')
  };
  return (
```

Продолжение Приложения А

```
<View style = {stylesheets.container}>
  <View style = {{alignItems: "center", top: '4%'}}>
    <GradientText style={stylesheets.style_FOCUS_2}>{login}</GradientText>
  </View>
  <View style = {stylesheets.view_played}>
    <Text style = {stylesheets.text_played}>
      Ваша статистика
    </Text>
  </View>
  <View style= {stylesheets.view_container}>
    <View style= {{flexDirection: "row", justifyContent: 'space-evenly'}}>
      <View>
    </View>
    <View style= {{flexDirection: 'row', justifyContent: 'space-around', top: '17%'}}>
      <TouchableOpacity onPress={()=>
        getMoviesMainWindow()
      }>
        <Image
          source={require('.././../assets/img/home_be.png')}
          style={stylesheets.image}
        />
      </TouchableOpacity>
      <TouchableOpacity>
        <Image
          source={require('.././../assets/img/user_be.png')}
          style={{width: null, resizeMode: 'contain', height: 50, width: 50, top: '4%'}}
        />
      </TouchableOpacity>
    </View>
  </View>
);
};
```

export default UserScreen;